

## Research paper

# An efficient framework for malicious network traffic detection using optimized deep learning techniques

Mukhtar Ahmed <sup>a,c</sup> , Jinfu Chen <sup>a,b</sup> ,\* , Ernest Akpaku <sup>a</sup> , Ajmal Latif <sup>c</sup>

<sup>a</sup> School of Computer Science and Communication Engineering, Jiangsu University, 301 Xuefu Road, Zhenjiang, 212013, Jiangsu, China

<sup>b</sup> Jiangsu Key Laboratory of Security Technology for Industrial Cyberspace, Jiangsu University, 301 Xuefu Road, Zhenjiang, 212013, Jiangsu, China

<sup>c</sup> Lasbela University of Agriculture Water and Marine Sciences, RCD Hwy Road, Uthal, 90150, Lasbela, Pakistan

## ARTICLE INFO

## Keywords:

Artificial intelligence  
Deep learning  
Malicious traffic detection  
Network security  
Zero-day attack  
Contrastive learning  
Multi-head attention

## ABSTRACT

The increasing sophistication of network attacks and the limitations of traditional machine learning-based detection methods pose significant challenges to modern cybersecurity. Existing approaches often depend heavily on labeled data and exhibit poor generalization across heterogeneous environments, limiting their effectiveness against emerging threats. To address these challenges, this paper introduces a novel framework for Malicious Network Traffic Detection (MNTD), designed to improve robustness and adaptability through deep learning methods. The model integrates Convolutional Neural Networks (CNNs), Bidirectional Long Short-Term Memory networks (BiLSTM), and Multi-Head Attention (MHA) mechanisms to capture both spatial and temporal dependencies in traffic. It further employs Adaptive Weighted Delay Velocity (AWDV) for hyperparameter optimization and contrastive learning to enhance feature discrimination, supported by an adaptive loss function and a regularized feature representation strategy to mitigate overfitting. The MNTD framework addresses a binary classification task, distinguishing between benign and malicious traffic. Experimental results demonstrate consistent state-of-the-art performance across four benchmark datasets. On the Canadian Institute for Cybersecurity Intrusion Detection System 2017 (CICIDS2017) dataset, it achieves 98.52% accuracy and a 98.98% F1-score. On the Canadian Institute for Cybersecurity Domain Name System (DNS) protocol Browser 2020 (CIRA-CIC-DoHBrw2020) dataset, it reaches 98.82% accuracy and 98.66% F1-score. For the Botnet Internet of Things (BoT-IoT) dataset, it obtains 98.65% accuracy and 98.40% F1-score, while on the University of New South Wales Network Benchmark 2015 (UNSW-NB15) dataset, it maintains 97.91% accuracy and 97.61% F1-score. This study demonstrates how artificial intelligence techniques can be effectively applied to cybersecurity applications, specifically malicious network traffic detection.

## 1. Introduction

The rapid expansion of the Internet has led to a substantial increase in both the volume and complexity of network traffic. This surge, coupled with the evolution of intricate network architectures, has expanded the attack surface for malicious actors to conduct sophisticated cyberattacks, such as network intrusions (Bautista et al., 2024) and the spread of disinformation across digital platforms (Fang et al., 2022). Unlike legitimate traffic, malicious network activity often demonstrates distinctive behavioral patterns, including repeated requests, resource exploitation, and anomalous payloads. These patterns frequently form highly correlated flows that reflect specific user or attacker behaviors, introducing intricate dependencies within network environments (Chen et al., 2023a). Consequently, accurately identifying and mitigating such activities remains a critical and complex challenge in cybersecurity.

Recent advances in deep learning (DL) have shown considerable promise in addressing this issue, particularly through models that can effectively capture the spatial and temporal dependencies present in network traffic (Fu et al., 2021; Ahmed et al., 2025a). DL architectures, including convolutional and recurrent neural networks (Ahmed et al., 2025b; Akpaku et al., 2025), are especially well-suited for analyzing the sequential nature of network flows in dynamic environments, such as social networks, financial systems, and communication infrastructures. These strengths have spurred extensive research into DL-based intrusion detection systems. Nonetheless, despite their potential, existing DL approaches continue to face notable limitations. In particular, many models struggle to generalize effectively to previously unseen or evolving attack types, especially in heterogeneous or real-world network conditions (Sun et al., 2020a). This limitation is often attributed to

\* Corresponding author.

E-mail address: [jinfuchen@ujs.edu.cn](mailto:jinfuchen@ujs.edu.cn) (J. Chen).

<https://doi.org/10.1016/j.engappai.2025.113592>

Received 17 April 2025; Received in revised form 20 September 2025; Accepted 16 December 2025

Available online 20 December 2025

0952-1976/© 2025 Elsevier Ltd. All rights reserved, including those for text and data mining, AI training, and similar technologies.

model overfitting and the dependency on large volumes of labeled data (Chou and Jiang, 2021), which are not always available in practical deployment scenarios (Lei et al., 2021). Although deep learning methods improve detection accuracy under controlled experimental settings, their real-world applicability remains constrained due to challenges such as overfitting to specific traffic patterns, sensitivity to hyperparameter selection, inability to generalize across heterogeneous datasets, and limited robustness to encrypted or evolving traffic flows.

To address these constraints, a robust malicious traffic detection system must support multi-level analysis—encompassing both aggregated traffic behavior and fine-grained examination of individual flows (Fu et al., 2024). Such a system must also ensure timely detection without incurring excessive processing delays, as real-time responsiveness is crucial for mitigating threats before significant damage occurs. Furthermore, computational efficiency must be maintained to avoid bottlenecks that could impact legitimate traffic. Traditional machine learning (ML) and DL approaches primarily rely on statistical features derived from flow-level attributes within fixed time windows (Zhang et al., 2018). Although effective in some scenarios, these techniques often falter in dynamic environments where diverse traffic patterns emerge. Overcoming this limitation requires a transition from purely statistical representations toward models that understand the structural and sequential characteristics of network flows. This includes examining packet-level sequences and aggregating flows into meaningful communication structures over specific temporal intervals (Ahmed et al., 2024).

While previous works have employed CNN-BiLSTM-Attention architectures for network traffic analysis, most of these studies primarily emphasize improving detection accuracy on individual benchmark datasets. However, they often overlook several critical aspects that affect real-world applicability. In particular, prior models tend to exhibit high sensitivity to hyperparameter settings, which limits their reproducibility and stability across heterogeneous environments. Moreover, they generally fail to generalize effectively to encrypted traffic such as DNS-over-HTTPS, and they struggle to maintain robustness when confronted with zero-day attacks or evolving traffic morphologies. These shortcomings highlight the need for a more comprehensive approach that not only improves classification performance but also enhances adaptability, scalability, and resilience against novel and obfuscated threats. In this context, the proposed MNTD framework extends beyond conventional CNN-BiLSTM-Attention pipelines by integrating CNN-based spatial feature extraction, BiLSTM-driven temporal modeling, multi-head attention for capturing long-range dependencies, and AWDV-driven automated hyperparameter optimization. Additionally, contrastive learning is incorporated to strengthen feature discrimination and improve generalization, enabling MNTD to handle adaptive and previously unseen attack patterns with greater reliability.

In response to these challenges, this paper introduces a novel framework for Malicious Network Traffic Detection (MNTD). The proposed framework combines Convolutional Neural Networks (CNN), Bidirectional Long Short-Term Memory (BiLSTM) (Chen et al., 2023b), and Multi-Head Attention (MHA) (Chen et al., 2023c) mechanisms to effectively capture both local patterns and global dependencies in network traffic. Furthermore, the model incorporates Adaptive Weighted Delay Velocity (AWDV) (Xu et al., 2022) and contrastive learning to enhance feature discrimination and improve generalization, enabling robust detection of zero-day and evolving threats. To further promote model stability, the architecture employs a regularized feature representation strategy and an adaptive loss function with dynamic thresholds, thereby mitigating overfitting and enhancing learning efficiency in diverse operational environments.

The primary contributions of this study are as follows:

- We present a novel framework for malicious network traffic detection, termed MNTD, which incorporates both local and global traffic flow analysis, as well as aggregate traffic details, to enhance detection capabilities.
- We integrate a multi-head self-attention mechanism into CNN and BiLSTM networks, enabling the model to assign greater weight to essential features. This improves the stability and detection efficiency of malicious network traffic methods.
- To mitigate overfitting in network environments with limited labeled data, we propose a robust feature representation strategy with integrated regularization techniques. This approach allows the model to maintain strong accuracy and reliability, even on smaller datasets. The improved adaptive loss function with dynamic thresholds further enhances the learning process, leading to better detection performance.
- Our model integrates AWDV and contrastive learning techniques to ensure high detection accuracy across various network environments. AWDV optimizes the model's hyperparameters and feature representation, while contrastive learning enhances generalization by learning robust feature representations. This combination makes the model resilient and broadly applicable to dynamic network conditions, even in the presence of limited labeled data.
- We conducted extensive evaluation experiments on benchmark datasets to demonstrate that leveraging structured information within network traffic enables the proposed model to achieve superior results compared to state-of-the-art methods.

The rest of the paper is organized as follows. Section 2 discusses related work in network traffic detection. Section 3 details the proposed model architecture and design. Section 4 presents the experimental setup and evaluation metrics. Sections 5 and 6 discuss the results and comparisons with baseline methods. Finally, Section 7 concludes the paper with future research directions.

## 2. Related works

This section reviews existing research in malicious network traffic detection, progressing from traditional machine learning methods to deep learning-based approaches, attention-enhanced architectures, and recent advances such as contrastive learning. We also situate the proposed MNTD framework within this broader context and highlight how it addresses persistent challenges in the field.

Prior studies span a wide spectrum, including traditional machine learning classifiers, hybrid CNN/RNN architectures augmented with attention, transformer-style models, contrastive learning approaches, and reinforcement learning for adaptive defenses. Despite their contributions, most of these works optimize either local spatial patterns or long-range temporal dependencies in isolation, assume fixed or static attention weights, or rely on manual hyperparameter tuning with limited search procedures. Such limitations reduce robustness under evolving traffic morphologies and hinder adaptability to emerging threats. Furthermore, many existing methods remain highly sensitive to class imbalance, resulting in brittle decision boundaries when evaluated on heterogeneous datasets (Chai et al., 2024). Consequently, their ability to generalize to obfuscated traffic or zero-day attack scenarios is often restricted.

### 2.1. Machine learning approaches

Early research on malicious network traffic detection primarily relied on traditional machine learning classifiers such as Decision Trees, Random Forests, Support Vector Machines (SVMs), and k-Nearest Neighbors (k-NN). These models were typically trained on flow-level statistical features extracted from datasets such as KDDCup'99 and NSL-KDD, with the goal of distinguishing between benign and attack traffic. While such methods demonstrated competitive performance on small or balanced datasets, they exhibited several critical limitations. First, their effectiveness was highly dependent on manual feature engineering, which restricted scalability and adaptability to new attack

types. Second, traditional ML classifiers often struggled with high-dimensional feature spaces and imbalanced traffic distributions, leading to degraded recall for minority attack classes.

Finally, these approaches lacked the ability to model complex temporal dependencies in traffic flows, a factor that is essential for detecting stealthy or evolving attack behaviors. These limitations motivated the transition toward deep learning-based methods capable of learning richer feature representations directly from raw or minimally processed traffic data.

## 2.2. Deep learning-based detection models

To overcome the limitations of traditional ML, researchers have adopted deep learning (DL) architectures for their automatic feature extraction and high representation capacity. CNNs have been used to capture spatial features in traffic payloads (Chen et al., 2022), while LSTM and BiLSTM models effectively model temporal dependencies (Ahmad et al., 2022; Elmasry et al., 2020). For example, Ahmad et al. (2022) benchmarked various neural models across multiple datasets, highlighting their benefits in intrusion detection tasks. Elmasry et al. (2020) combined DNN, LSTM, and DBN with dual PSO pretraining to increase accuracy. Chen et al. (2022) employed Deep Belief Networks with LSTM classifiers, yet scalability and real-time applicability remained challenges. These works paved the way for hybrid modeling techniques.

Recent studies have explored deep reinforcement learning techniques for the detection of malicious traffic, particularly in environments where dynamic attacks and zero-day attacks are prevalent. Yu et al. (2024b) proposed a DQN-based intrusion detection framework that integrates stochastic game modeling with optimized hyperparameter tuning to enhance detection capabilities. By formulating the defense mechanism as a sequential decision-making problem under uncertainty, the model effectively adapts to adversarial behaviors. Building on this, Yu et al. (2024c) introduced an open-set intrusion detection solution using Deep Q-Networks, capable of identifying and responding to previously unseen attack types—an essential requirement in real-world deployments. Shen et al. (2024) developed a heuristic DQN approach specifically targeted at zero-day threats in edge-based environments. Their method combines temporal analysis with edge computing to improve responsiveness against stealthy intrusions. In a related effort, Shen et al. (2023) presented a malware containment strategy leveraging joint differential game theory and double DQNs to suppress malware propagation. More recently, Zhu et al. (2025) proposed RT-A3C, a real-time asynchronous advantage actor-critic model designed for edge-enabled defense system. This approach enhances performance over traditional DQNs by reducing detection latency and accelerating policy convergence, making it suitable for time-sensitive industrial environments.

Software-Defined Networking (SDN) has emerged as a critical paradigm in modern network infrastructures, yet it remains highly vulnerable to Distributed Denial of Service (DDoS) attacks due to its centralized control plane. Recent works have explored deep learning-based solutions for SDN DDoS detection, including CNN-based classifiers applied to OpenFlow statistics, and reinforcement learning approaches that dynamically adjust mitigation strategies at the controller level. Such methods demonstrate the importance of real-time and low-latency detection in SDN environments. Given that the proposed MNTD achieves sub-millisecond inference latency (Table 9), it is well-suited for deployment in SDN-based security frameworks where rapid detection and response are essential. CNN models capture packet/byte locality but lack explicit temporal context; LSTM/BiLSTM-only models learn sequence dynamics but can underutilize local payload structure. Hybrid CNN-(Bi)LSTM stacks address parts of this gap, yet without principled attention and regularization, they remain sensitive to imbalance and overfit to dataset-specific artifacts.

Several studies have explored hybrid models for capturing spatial and temporal patterns in traffic data. Tao et al. (2020) proposed

AGG, an attention-based hybrid model that combines Graph Convolutional Networks (GCN) and Gated Recurrent Units (GRU) to capture spatial correlations and temporal dependencies, with an attention mechanism emphasizing the most relevant features. Lu et al. (2020) introduced RF-CWRNN, a hybrid framework that integrates Clockwork RNNs (CWRNN) to model multi-scale temporal dependencies with Random Forests (RF) to select optimal input time windows, achieving improved efficiency and accuracy in multi-lane traffic speed forecasting. Similarly, Gu et al. (2019) developed a CNN-based model for classifying cycling maneuvers in urban environments, effectively extracting spatial features from human, road, and traffic-related data to outperform classical machine learning approaches. While these studies demonstrate strong capabilities in modeling spatial-temporal patterns for numeric traffic forecasting or behavior classification, they are not designed for detecting anomalies in network traffic. In contrast, the proposed MNTD framework leverages CNNs for spatial feature extraction, BiLSTMs for bidirectional temporal modeling, multi-head attention to capture long-range dependencies, and contrastive learning to build robust feature representations, specifically targeting malicious network traffic detection across heterogeneous and encrypted datasets.

Recent research has increasingly explored image-based malware detection methods that transform binary executables into grayscale or RGB images for classification. Kumar and Kumar (2024) introduced an image-based malware detection framework for IIoT environments using CNNs with a two-level autoencoder and SDN honeypots, achieving 98.5% accuracy on the Mallmg dataset. Similarly, Kumar et al. (2024) proposed IMCNN, which leverages transfer learning and ensemble learning on honeypot-enabled networks, reporting 99.36% accuracy on Mallmg and 92.11% on real-world malware. Kumar and Panda (2023) further advanced this direction with SDIF-CNN, stacking deep image features from fine-tuned CNNs (VGG16, VGG19, ResNet50, InceptionV3) combined with feature selection and classification, achieving 98.55% accuracy on Mallmg and 94.78% on real-world datasets. Kumar and Janet (2022) proposed DTMIC, a transfer learning-based framework converting executables into grayscale images and applying pre-trained CNN architectures, attaining 98.92% on Mallmg and 93.19% on Microsoft BIG dataset. Alongside these CNN-based approaches, Kumar et al. (2022) developed a framework using stacked local and global textural features with machine learning classifiers, while Kumar and Janet (2021) introduced MTIS, a hybrid learning architecture combining CNNs and handcrafted descriptors such as LBP, DSIFT, and GLCM. These methods demonstrate high accuracy and robustness against obfuscation and packing, yet they remain fundamentally constrained by static binary-to-image transformations and offline feature engineering.

In contrast, the proposed MNTD framework directly models raw network traffic rather than transformed binaries, enabling real-time anomaly detection in dynamic environments. By integrating CNNs for spatial feature extraction, BiLSTMs for temporal dependency modeling, multi-head attention for capturing long-range interactions, AWDV-based optimization, and contrastive learning, MNTD addresses the challenges of zero-day threats, domain shifts, and encrypted traffic. This design allows MNTD to move beyond static visualization approaches, offering a scalable and adaptive solution for malicious network traffic detection across heterogeneous and evolving datasets.

## 2.3. Multi-head attention in malicious network traffic detection

Hybrid frameworks emerged to combine the strengths of CNNs and RNNs, often enhanced with attention mechanisms for feature prioritization. Attention allows models to dynamically focus on informative parts of the input, significantly improving interpretability and performance. Yu et al. (2024a) introduced a GWO-CNN-BiLSTM model, using Grey Wolf Optimization for parameter tuning. Wang (2024) proposed a WGAN-CNN-BiLSTM model with attention in a cloud edge deployment scenario. Zhang et al. (2023b) used multi-head attention

to enhance temporal feature extraction over BiLSTM outputs. Although these approaches advanced detection accuracy, they generally lacked robustness to evolving attack behaviors.

Attention-augmented hybrids such as GWO-CNN-BiLSTM and WGAN-CNN-BiLSTM use meta-heuristics or adversarial components for tuning and class balancing, but typically rely on fixed attention placement and lack hierarchical, flow-level reweighting; as a result, they improve accuracy yet remain vulnerable to domain shifts and evolving behaviors. Reported baselines in our study (e.g., AB-BiLSTM, CNN-LSTM, DCNN-BiLSTM) confirm that even attention-enabled stacks can exhibit higher FPR or slower convergence when attention is not paired with stronger optimization and representation regularization.

#### 2.4. Recent advancements: Transformers, contrastive learning

Recent literature has focused on improving model generalization through architectural innovation and robust training strategies. Transformer-based models have attracted attention because of their ability to model long-range dependencies through self-attention mechanisms. Zhang et al. (2023a) proposed a transformer-based intrusion detection system leveraging positional encoding. Although effective for sequence learning, it lacked contrastive regularization for generalization. Nguyen and Lee (2024) introduced a multiview cross-attention learning framework that combines payload and statistical features, achieving strong results but at the cost of computational efficiency.

Contrastive learning has shown significant promise in improving feature robustness. Li et al. (2024) proposed GraphCL-ID, a graph-based contrastive learning model for the detection of anomalous IoT. Hu et al. (2023) introduced CLNet, which learns discriminatory attack embeddings through self-supervised contrastive training. However, these models often rely on graph structures or are designed primarily for pre-training, limiting direct integration into flow-based classification pipelines.

Several recent models share architectural similarities with our proposed framework. Kamal and Mashaly (2024) proposed a Transformer-CNN hybrid for intrusion detection, incorporating resampling strategies such as SMOTE and ADASYN. Although highly effective, their method lacked hierarchical attention and advanced regularization techniques. Naeem et al. (2025) presented an attention-enhanced CNN-BiLSTM model specifically for IoT botnet detection. However, the model was designed for a narrow use case and was unable to be evaluated in more diverse and complex datasets.

Transformer variants improve long-range dependency modeling but often increase computational cost and may omit contrastive regularization, reducing cross-dataset robustness. Contrastive and domain-adaptive methods strengthen representation invariance, yet many either depend on graph constructions or serve primarily for pretraining, complicating direct flow-level integration for low-latency inference.

Deep reinforcement learning approaches are promising for mitigation policy learning and open-set recognition, but they address a different layer of the stack (response/containment). They typically complement, rather than replace, high-throughput flow classifiers required on the critical path.

With the increasing prevalence of encrypted communication protocols such as TLS and DNS-over-HTTPS (DoH), malicious traffic is increasingly concealed within encrypted flows. Recent studies have applied transformer-based models to TLS packet sequences for encrypted traffic classification, while meta-learning approaches have been employed for few-shot detection of zero-day encrypted threats. These approaches demonstrate promising adaptability but often suffer from high computational costs (see Table 1). In this context, the proposed MNTD framework — evaluated on four benchmark datasets — leverages multi-head attention and contrastive learning to achieve robust detection, thereby offering a practical and efficient solution for encrypted traffic analysis.

From this study, it shows that many existing approaches are limited by rigid feature extraction mechanisms, domain specificity, poor generalization, and computational efficiency to unseen threats. In contrast, our proposed MNTD framework employs a unified architecture comprising CNN, BiLSTM, and multi-head attention to jointly capture local spatial features, temporal dynamics, and cross-flow dependencies. The integration of AWDV tuning further enhances the model's sensitivity to traffic behavior timing, while contrastive learning improves feature robustness. An adaptive loss function with regularization enhances the stability of training and reduces overfitting. Together, these innovations enable MNTD to outperform state-of-the-art baselines in terms of accuracy, generalizability, and computational efficiency. To validate these contributions, our evaluation (Section 5) includes a comparison with recently proposed attention-based, transformer-based, and contrastive learning models across various benchmark datasets. This comparative analysis highlights the practical effectiveness of MNTD in diverse network environments.

*How MNTD advances the state-of-the-art:* MNTD integrates packet-local feature extraction (CNN), bidirectional temporal modeling (BiLSTM), and cross-flow dependency capture (MHA) within a unified architecture. This is coupled with (i) AWDV-based hyperparameter optimization to mitigate brittle configurations and accelerate stable convergence, and (ii) an adaptive loss with dynamic thresholding and contrastive regularization to enforce tighter decision boundaries under class imbalance. In doing so, MNTD directly addresses the limitations identified above—namely, reliance on fixed attention placement, static hyperparameter search, and weak generalization across heterogeneous datasets.

*Practical implications:* The combination of multi-head attention, adaptive loss, and AWDV-guided hyperparameter tuning produces a detector that is resilient to obfuscation and dataset variability, while maintaining low-latency inference suitable for near real-time deployment. This positions MNTD as a practical and generalizable alternative to prior attention-based hybrids and computationally heavy transformer architectures for binary malicious traffic detection.

### 3. Proposed method

#### 3.1. MNTD framework

The MNTD (Malicious Network Traffic Detection) framework is a state-of-the-art deep learning model designed to detect malicious network traffic with high accuracy and robustness. It integrates CNNs, BiLSTM networks, and MHA mechanisms to capture both local spatial features and long-range temporal dependencies in network traffic.

CNNs are employed to extract packet-level characteristics, such as headers and payloads, while BiLSTMs model the sequential behavior of network flows considering both historical and forward context. The MHA mechanism enables the model to take care of multiple aspects of the input simultaneously, allowing it to identify both prominent and subtle anomalies, as illustrated in Fig. 1.

The framework also incorporates regularization techniques to mitigate overfitting, along with an adaptive loss function based on dynamic thresholds to enhance learning stability. To optimize the hyperparameters of the model, the AWDV algorithm is applied, allowing the framework to adapt effectively to evolving network conditions and threat patterns.

MNTD operates in two distinct phases: a training phase, where shared weights and a joint loss function guide the model in learning discriminative embeddings, and a testing phase, where unseen traffic samples are classified as benign or malicious based on the learned representations. This comprehensive design ensures high detection performance, robustness, and scalability in dynamic and high-throughput network environments.

The integration of CNN, BiLSTM and MHA in the proposed MNTD framework is not arbitrary, but grounded in their complementary

**Table 1**  
Comparative summary of prior network traffic detection methods.

| Reference                                                                                      | Dataset(s)                  | Architecture/Method                  | Limitations                                                                                                                    |
|------------------------------------------------------------------------------------------------|-----------------------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Tao et al. (2020)                                                                              | Traffic speed datasets      | AGG: GCN + GRU with attention        | Designed for numeric forecasting; not applicable to malicious network traffic; limited to continuous spatial-temporal patterns |
| Lu et al. (2020)                                                                               | Multi-lane traffic datasets | RF-CWRNN: Clockwork RNN + RF         | Optimized for traffic prediction; lack of anomaly detection; numeric-focused                                                   |
| Gu et al. (2019)                                                                               | Cycling maneuver datasets   | CNN-based spatial feature extraction | Targets physical behavior classification; not designed for network traffic anomalies                                           |
| Ahmad et al. (2022)                                                                            | Various IDS datasets        | DNN, LSTM/BiLSTM                     | Effective temporal modeling; lacks integration of spatial patterns; may overfit to dataset-specific artifacts                  |
| Chen et al. (2022)                                                                             | IDS datasets                | DBN + LSTM                           | Scalability and real-time applicability remain challenging                                                                     |
| Yu et al. (2024a)                                                                              | IDS datasets                | GWO-CNN-BiLSTM                       | Attention placement is fixed; vulnerable to domain shifts; meta-heuristic tuning required                                      |
| Wang (2024)                                                                                    | Cloud-edge IDS datasets     | WGAN-CNN-BiLSTM                      | Addresses class imbalance; fixed attention placement; sensitive to evolving attacks                                            |
| Zhang et al. (2023b)                                                                           | IDS datasets                | BiLSTM + Multi-head Attention        | Captures long-range temporal dependencies; limited spatial feature modeling                                                    |
| Ren et al. (2019)                                                                              | IDS datasets                | AB-BiLSTM                            | Improves long distance dependency modeling; attention is not fully adaptive; may overfit dataset specific patterns             |
| Sun et al. (2020b)                                                                             | IDS datasets                | CNN-LSTM                             | Captures both spatial and sequential features; lacks hierarchical attention; sensitive to imbalance                            |
| Hnamte and Hussain (2023)                                                                      | IDS datasets                | DCNN-BiLSTM                          | Deep CNN + BiLSTM; may overfit without regularization; attention not explicitly integrated                                     |
| Kumar and Kumar (2024), Kumar et al. (2024), Kumar and Panda (2023) and Kumar and Janet (2022) | Mallimg, Microsoft BIG      | CNN/transfer learning/autoencoders   | High accuracy for image based malware detection; limited by static binary-to-image transformations; offline processing         |
| Li et al. (2024) and Hu et al. (2023)                                                          | IoT/IDS anomaly datasets    | Graph-based contrastive learning     | Pretraining or graph-dependent; not directly flow-level; low real-time applicability                                           |

strengths for the detection of malicious traffic. CNN layers specialize in extracting local spatial features, such as packet-byte patterns or protocol-specific headers, which are often indicative of localized malicious behaviors. However, CNNs alone cannot capture temporal evolution. To address this, BiLSTM layers are incorporated to model bidirectional sequential dependencies within network flows, enabling the model to understand how malicious behavior unfolds over time. However, BiLSTM may treat all time steps with similar importance, potentially overlooking critical anomalies. Thus, MHA is integrated to dynamically reweight temporal features, allowing the model to focus on the most relevant parts of the input sequence. Each attention head captures different aspects of the sequence, enhancing the model's discriminative ability (see Fig. 2). This synergy enables the framework to comprehensively learn both spatial and temporal dynamics, adaptively highlighting salient traffic patterns that might indicate stealthy or evolving cyber threats.

### 3.1.1. Model input and transformation dimensions

To enhance clarity regarding the data flow and internal transformations of the proposed MNTD framework, we provide a detailed description of the input dimensions at each stage of the architecture. The input to the model is a communication channel  $C_{cc}$ , which aggregates multiple temporally correlated network flows. Each network flow  $f$  is composed of both statistical features  $x_{stat}$  and payload features  $x_{payload}$ .

- **Input Representation:** The combined input feature tensor is of shape  $(N_f, N_p, M)$ , where  $N_f$  is the number of flows in the communication channel,  $N_p$  is the number of packets per flow (e.g., 30), and  $M$  is the number of bytes per packet retained after truncation or padding (e.g., 64).
- **CNN Layer:** A 1D convolution is applied along the byte dimension of each packet, enabling localized spatial feature extraction. The CNN input per flow is  $(N_p, M)$ . After Conv1D with  $F$  filters and kernel size  $K$ , followed by max pooling with stride  $S$ , the output shape becomes  $(N'_p, F)$ , where  $N'_p = \frac{N_p - K}{S} + 1$ .

- **BiLSTM Layer:** The CNN outputs are flattened into flow-level representations and processed as temporal sequences. The BiLSTM input shape is  $(N_f, d_{cnn})$ , where  $d_{cnn}$  is the flattened CNN feature dimension. The BiLSTM outputs hidden states of shape  $(N_f, 2H)$ , where  $H$  is the number of LSTM units per direction.
- **Multi-Head Attention:** The attention module receives the BiLSTM output  $(N_f, 2H)$  and projects it into query, key, and value matrices. With  $h$  heads and head dimension  $d_k$ , the intermediate representation becomes  $(N_f, h, d_k)$ . After attention aggregation and linear transformation, the output remains  $(N_f, 2H)$ .
- **Fully Connected Layers:** The attention-enhanced features are passed through one or more dense layers for classification. After global pooling, the input to the final dense layer is  $(2H)$ , and the output dimension is  $(C)$ , where  $C$  denotes the number of classes (e.g.,  $C = 2$  for binary classification of benign vs. malicious traffic).

This dimensional flow illustrates how the model transforms raw network traffic into compact, high-level representations for effective classification. It also clarifies how MNTD captures both local dependencies within flows and global dependencies across flows in a communication channel (see Fig. 4).

### 3.2. Convolutional neural networks

CNNs (Anitha et al., 2023) are highly effective in identifying local patterns and spatial relationships in data through the use of convolutional layers. In the context of network traffic detection, CNNs are employed to extract local features from raw traffic data, such as packet headers and payloads. These features are critical for identifying patterns associated with malicious activities as they capture localized characteristics that can indicate specific types of attacks.

#### Feature extraction process

**Input Data:** Raw network traffic data is fed into the model as input. This data typically includes packet-level information, such as headers and payloads, which are processed to extract meaningful features.

**Convolutional Layers:** Multiple convolutional layers are applied to the input data, utilizing filters (kernels) to extract local features. Each filter scans the input data to detect specific patterns, such as byte sequences or packet structures, that are indicative of malicious behavior. The convolution operation is mathematically expressed as:

$$\text{Feature Map}_{ij} = \sum_k (X * W_k)_{ij} + b_k \quad (1)$$

where  $X$  is the input data,  $W_k$  is the  $k$ th filter,  $*$  denotes the convolution operation,  $b_k$  is the bias term, and  $\text{Feature Map}_{ij}$  is the value at position  $(i, j)$  in the output feature map.

**Activation Function:** Following the convolution operation, a non-linear activation function is applied to introduce non-linearity into the model. In the CNN component of our framework, we employ the Rectified Linear Unit (ReLU) due to its computational efficiency and empirically strong performance in convolutional architectures:

$$\begin{aligned} \text{Activated Feature Map}_{ij} &= \text{ReLU}(\text{Feature Map}_{ij}) \\ &= \max(0, \text{Feature Map}_{ij}) \end{aligned} \quad (2)$$

ReLU expedites training and reduces vanishing-gradient effects, while also contributing to sparse activations that improve convergence and reduce overfitting.

**Pooling Layers:** Pooling layers are applied to reduce the spatial dimensions of feature maps while preserving the most informative features. This not only reduces computational cost but also increases robustness to local variations and noise in packet sequences. The pooling operation, typically max pooling, is defined as:

$$\text{Pooled Feature Map}_{ij} = \max_{(m,n) \in P_{ij}} \text{Activated Feature Map}_{mn} \quad (3)$$

where  $P_{ij}$  is the pooling region centered at  $(i, j)$ .

**Flattening and Integration:** After several convolutional and pooling layers, the feature maps are flattened into fixed-size vectors, which are then forwarded to the BiLSTM and attention modules for temporal and dependency modeling before reaching the fully connected layers for final classification.

#### Key contributions of the CNN component

**Local Pattern Detection:** The CNN component excels at capturing localized patterns in network traffic, such as specific byte sequences or packet structures, which are often indicative of malicious activities.

**Dimensionality Reduction:** Pooling layers reduce the spatial dimensions of feature maps, preserving critical features while minimizing computational overhead and improving generalization.

**Non-Linearity and Robustness:** The use of the ReLU activation function introduces non-linearity, enabling the model to learn complex patterns efficiently while mitigating the vanishing gradient problem. ReLU further enhances robustness by allowing sparse activation, which reduces overfitting and accelerates convergence.

#### 3.3. Bidirectional long short-term memory

Following CNN layers, the feature vector is passed into the BiLSTM network (Anitha et al., 2023). LSTMs are designed to learn long-term dependencies in sequential data, making them particularly effective for analyzing temporal patterns in network traffic. The bidirectional aspect of BiLSTM enhances this capability by allowing the model to incorporate context from both the previous and subsequent states in the sequence. This dual-directional processing is crucial for detecting complex attack patterns that may only be identifiable by examining relationships across multiple network flows.

#### LSTM cell structure

Each LSTM cell consists of three gates that govern the flow of information:

- **Input Gate ( $i_t$ ):** Controls the extent to which new information is added to the cell state.
- **Forget Gate ( $f_t$ ):** Determines how much of the previous cell state is retained or discarded.
- **Output Gate ( $o_t$ ):** Regulates the output of information from the cell state to the hidden state.

The LSTM process is mathematically expressed as follows:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (4)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (5)$$

$$\tilde{C}_t = \text{ELU}(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (6)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (7)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (8)$$

$$h_t = o_t * \text{ELU}(C_t) \quad (9)$$

where  $\sigma$  denotes the sigmoid activation function, ELU refers to the Exponential Linear Unit activation function,  $*$  denotes element-wise multiplication,  $W$  represents weight matrices,  $b$  represents bias vectors,  $x_t$  is the input at time step  $t$ ,  $h_{t-1}$  is the hidden state of the previous time step,  $C_t$  is the cell state at time step  $t$ , and  $\tilde{C}_t$  is the candidate cell state.

In the BiLSTM component, we employ the Exponential Linear Unit (ELU) activation function. ELU has the advantage of allowing negative outputs, which helps reduce bias shift and improves the stability of the learning process in recurrent layers. Unlike ReLU, which outputs zero for all negative inputs, ELU provides smooth gradients for negative values, thereby enhancing the model's ability to capture subtle temporal variations in malicious traffic and sequential data. This choice complements the use of ReLU in the CNN layers, where fast convergence and sparse activations are beneficial. Together, the combination of ReLU in the convolutional layers and ELU in the BiLSTM layers allows the framework to balance efficiency with robust temporal feature learning.

#### Bidirectional processing

A BiLSTM consists of two LSTM networks: one processes the input sequence in the forward direction, while the other processes it in the backward direction. The forward and backward hidden states are computed as:

$$h_t^{\text{forward}} = \text{LSTM}(x_t, h_{t-1}, C_t) \quad (10)$$

$$h_t^{\text{backward}} = \text{LSTM}(x_t, h_{t+1}, C_t) \quad (11)$$

The final hidden state at each time step is obtained by concatenating the forward and backward hidden states:

$$h_t = [h_t^{\text{forward}}; h_t^{\text{backward}}] \quad (12)$$

where  $[\cdot]$  denotes concatenation. The BiLSTM network generates a forward hidden state  $h_t^{\text{forward}}$  and a backward hidden state  $h_t^{\text{backward}}$  at each time step  $t$ . The final hidden state of the BiLSTM network serves as the output of the spatial-temporal feature extractor, denoted  $e_{fl} \in \mathbb{R}^d$ , where  $d$  represents the size of the hidden state.

#### Dropout for regularization

To reduce overfitting and improve generalization, dropout layers are incorporated into the BiLSTM network (Ren et al., 2019). Dropout randomly deactivates a fraction of neurons during training, preventing the model from becoming overly reliant on specific features and enhancing its ability to generalize to unseen data.

### Key contributions of the BiLSTM component

- **Temporal Dependency Modeling:** The BiLSTM component effectively captures long-term dependencies in sequential network traffic data, enabling the detection of complex attack patterns that span multiple time steps.
- **Bidirectional Context:** By processing data in both forward and backward directions, the BiLSTM network incorporates contextual information from past and future states, improving its ability to identify subtle anomalies.
- **Regularization:** The use of dropout layers enhances the model's robustness and generalization capabilities, reducing the risk of overfitting.

### 3.4. Network flow-level attention layer

The proposed MNTD model employs a MHA mechanism (Chen et al., 2023c) to enhance feature representation by capturing both intra-flow and inter-flow dependencies. A communication channel is modeled as a set of temporally correlated network flows, where each flow comprises a sequence of packets encoded into fixed-length vectors. The processing pipeline integrates multiple deep learning components to capture spatial, temporal, and contextual relationships within and between flows.

Initially, each network flow is passed through a single 1D convolutional layer, which extracts local spatial features from packet-level byte sequences. These CNN-extracted features are then fed into BiLSTM layers to model the temporal dependencies within the packet sequences of each flow. The BiLSTM outputs a sequence embedding for each network flow, which captures contextualized information over time.

To further enhance the model's capacity to capture dependencies between multiple flows within a communication channel, we apply a multi-head attention mechanism at the flow level. Specifically, the outputs of the BiLSTM layers for all flows within a channel are aggregated and passed to the MHA module, which computes attention scores across flows to identify significant temporal and semantic relationships. This attention-based aggregation generates a global feature embedding that represents the entire communication channel. The attention mechanism is defined as follows:

$$K = X_{fl} W_K \quad (13)$$

$$Q = X_{fl} W_Q \quad (14)$$

$$V = X_{fl} W_V \quad (15)$$

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V \quad (16)$$

Here,  $X_{fl} \in \mathbb{R}^{(N_p+1) \times M}$  represents the BiLSTM output feature matrix, where  $N_p$  denotes the number of packets and  $M$  is the embedding dimension. The matrices  $W_K$ ,  $W_Q$ , and  $W_V$  are trainable parameters that project the input into key, query, and value spaces. The scaled dot-product attention computes weighted interactions across flows within a channel, enabling the model to focus on flows indicative of malicious behavior.

By applying MHA after the CNN and BiLSTM layers, the model effectively combines local spatial, temporal, and global inter-flow features. This structure ensures robust representation learning, allowing MNTD to detect subtle anomalies and adapt to diverse network environments. The integration of attention mechanisms at the flow level significantly improves generalization and detection accuracy in challenging multi-flow scenarios.

### 3.5. Adaptive weighted delay velocity

Adaptive Weighted Delay Velocity (AWDV) (Xu et al., 2022) is an advanced optimization algorithm designed to overcome the limitations of traditional Particle Swarm Optimization (PSO), such as premature convergence and local optima entrapment. AWDV enhances the standard PSO framework by introducing an adaptive weighted delay mechanism that dynamically adjusts the velocity and position updates of particles, improving exploration of the search space and preventing stagnation in suboptimal regions.

In this study, AWDV is employed for hyperparameter optimization of the proposed MNTD model (Song et al., 2024). The updates for the  $i$ th particle are defined as follows:

$$V_i(t+1) = W_i(t) + \alpha (P_{best} - W_i(t)) + \beta (G_{best} - W_i(t)) + \gamma D_i(t) \quad (17)$$

$$X_i(t+1) = X_i(t) + V_i(t+1) \quad (18)$$

Here,  $V_i(t+1)$  and  $X_i(t+1)$  denote the updated velocity and position of the  $i$ th particle at iteration  $t+1$ .  $W_i(t)$  represents the particle's current weight influencing its movement, while  $\alpha$  and  $\beta$  control the contributions of the personal best ( $P_{best}$ ) and global best ( $G_{best}$ ) positions. The term  $\gamma D_i(t)$  corresponds to the adaptive weighted delay component, which refines particle velocity updates to enhance exploration and exploitation balance.

The adaptive delay term  $D_i(t)$  enables AWDV to effectively adapt to changing network traffic patterns and evolving attack vectors, which is critical for robust detection of malicious activities in dynamic and heterogeneous network environments.

The coefficients  $\alpha = 0.9$  and  $\beta = 0.5$  were determined through empirical tuning and prior recommendations (Xu et al., 2022). A higher  $\alpha$  maintains exploration momentum, while  $\beta = 0.5$  provides a balanced influence between personal and global best positions, ensuring stable convergence. Preliminary experiments validated that these settings lead to optimal hyperparameter tuning and improved detection performance across all evaluated datasets.

By integrating the exploration capabilities of PSO with the adaptive delay mechanism of AWDV, the proposed approach achieves efficient and robust hyperparameter optimization, enhancing the overall accuracy and reliability of the MNTD framework.

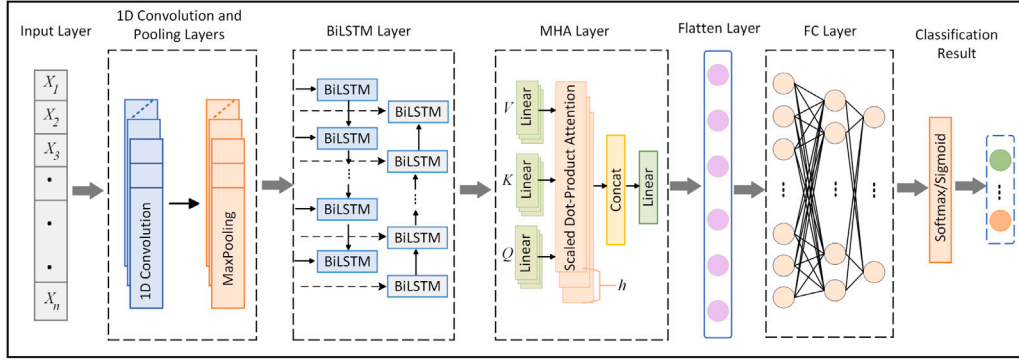
### 3.6. Contrastive learning

Contrastive learning is a self-supervised representation learning technique that aims to maximize similarity between positive pairs while minimizing similarity between negative pairs in the feature space. The key idea is to bring semantically similar samples closer and push dissimilar samples apart, thereby learning more discriminative feature embeddings without heavy reliance on labeled data.

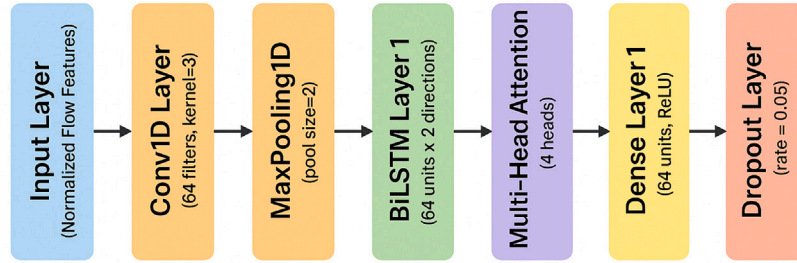
In the context of malicious network traffic detection, contrastive learning enhances the ability of the proposed MNTD model to distinguish between benign and malicious flows. By learning feature representations that remain robust under varying network conditions, the model generalizes effectively across diverse environments and is better equipped to detect unknown or zero-day attacks, where labeled data are often scarce.

The contrastive learning process involves three main steps:

- **Positive and Negative Pair Construction:** For each anchor sample (e.g., a network flow), a positive sample is selected from the same class, while negative samples are drawn from different classes. This creates the basis for contrastive comparisons.
- **Feature Embedding:** The anchor, positive, and negative samples are passed through the feature extractor (e.g., CNN or BiLSTM) to obtain their embeddings in the feature space.



**Fig. 1.** Architecture of the proposed MNTD model for network traffic detection. The input to the model is a communication channel composed of multiple flows, each represented by packet-level payload bytes and statistical features. A 1D convolutional layer extracts localized spatial features from packet bytes, while a BiLSTM layer models temporal dependencies across flows. Multi-head attention is then applied to capture cross-flow dependencies and emphasize critical patterns. Finally, dense layers with an adaptive loss function produce binary predictions of benign or malicious traffic. This design integrates spatial, temporal, and attention-based representations with AWDV-driven hyperparameter optimization and contrastive learning, enabling robust and generalizable detection across diverse network environments.



**Fig. 2.** Schematic flowchart of the proposed MNTD model architecture. The framework consists of a single Conv1D layer with max pooling for local feature extraction, two stacked BiLSTM layers for temporal dependency modeling, a multi-head attention mechanism for focusing on critical flow patterns, and fully connected layers with dropout before the final softmax output.

- **Contrastive Loss Optimization:** The loss function enforces closeness of anchor-positive embeddings while separating anchor-negative embeddings. It is formulated as:

$$\mathcal{L}_{\text{contrastive}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\text{sim}(f_{\theta}(x_i^a), f_{\theta}(x_i^p)) / \tau)}{\sum_{j=1}^N \exp(\text{sim}(f_{\theta}(x_i^a), f_{\theta}(x_j^n)) / \tau)} \quad (19)$$

where  $N$  is the number of samples in a batch,  $x_i^a$ ,  $x_i^p$ , and  $x_i^n$  represent the anchor, positive, and negative samples, respectively, and  $f_{\theta}$  denotes the feature extractor with parameters  $\theta$ . The function  $\text{sim}(\cdot, \cdot)$  measures similarity (typically cosine similarity), and  $\tau$  is a temperature parameter controlling the distribution sharpness.

By optimizing Eq. (19), the model maps similar flows closer together while pushing dissimilar flows farther apart. This strengthens the discriminative capability of the learned features and improves detection of subtle anomalies. Furthermore, the integration of contrastive learning enables effective use of unlabeled data, ensuring robust performance across dynamic and evolving cybersecurity environments.

### 3.7. Triplet sampling strategy for contrastive learning

To enhance the effectiveness of contrastive learning within the MNTD framework, a structured triplet sampling strategy is applied to generate anchor, positive, and negative flow samples.

**Positive sample selection:** For each anchor flow, a positive sample is drawn from the same class label (either benign or malicious). To ensure semantic closeness, cosine similarity between feature vectors is computed, and only samples with a similarity score above a defined threshold (e.g., 0.5) are selected. This guarantees that positive flow

samples are not only class-consistent but also structurally similar in the feature space.

**Negative sample selection:** Negative flow samples are chosen from the class opposite to that of the anchor. To maximize contrast in the learned representation, preference is given to samples exhibiting low cosine similarity to the anchor (e.g., below 0.3). This encourages the embeddings of dissimilar traffic flows to be pushed farther apart in the feature space.

**Temporal proximity consideration:** To preserve contextual relevance in datasets with sequential flow patterns, a temporal filter is applied to ensure that anchor and positive flow samples are drawn from nearby time windows. This constraint is particularly important for capturing subtle variations in evolving attack behaviors.

By integrating semantic, structural, and temporal criteria, the proposed triplet sampling strategy improves the generalization of learned embeddings and enhances the model's ability to discriminate between benign and malicious traffic, even in complex and dynamically evolving network environments (see Fig. 3).

### 3.8. Regularization and adaptive loss function

Regularization and adaptive loss functions are essential components of the proposed MNTD framework, designed to enhance robustness, prevent overfitting, and ensure effective generalization across diverse network environments. These mechanisms stabilize the learning process, particularly in scenarios with limited or noisy labeled traffic data, thereby improving detection performance.

### 3.8.1. Regularization

Regularization strategies are employed to prevent the model from overfitting to training data, which is a common risk due to the high-dimensional and heterogeneous nature of network traffic. In the proposed model, two primary regularization techniques are applied:

- **L2 Regularization (Weight Decay):** This approach penalizes large weight values by adding a quadratic penalty term to the loss function, which encourages smaller and more evenly distributed weights. The L2 penalty term is formulated as:

$$\mathcal{L}_{L2} = \lambda \sum_{i=1}^n \|w_i\|^2 \quad (20)$$

where  $w_i$  denotes the model parameters and  $\lambda$  controls the strength of the penalty.

- **Dropout:** Dropout stochastically deactivates a fraction of neurons during training, reducing co-adaptation and forcing the model to learn more robust and distributed feature representations. In the proposed framework, dropout is applied to the fully connected layers with a dropout rate of  $p$  (e.g.,  $p = 0.05$ ).

By combining weight decay and dropout, the proposed model achieves a balance between accurately fitting the training data and maintaining the ability to generalize effectively to unseen traffic, which is crucial for reliable detection in dynamic and evolving network environments.

---

#### Algorithm 1 Training procedure for the MNTD framework.

---

**Require:** Network traffic dataset  $D$ , Hyperparameters:  $N_{epoch}$ ,  $N_{batch}$ ,  $\lambda_1$ ,  $\lambda_2$ , learning rate  $\eta$

**Ensure:** Trained model parameters  $\theta^*$

```

1: Initialization: Initialize model parameters  $\theta$  for MNTD optimization.
2: for  $epoch \leftarrow 1$  to  $N_{epoch}$  do
3:   for each mini-batch  $B \in D$  do
4:     Step 1: Feature extraction
5:     Extract statistical features  $X_{stat}$  and payload features  $X_{payload}$ .
6:     Construct communication channels  $C_{cc}$  by aggregating temporally
       correlated network flows.
7:     Normalize the extracted features using min-max scaling.
8:     Step 2: Feature representation
9:     Pass  $C_{cc}$  through CNN layers to extract local spatial features.
10:    Process CNN output with BiLSTM to model temporal dependencies.
11:    Apply MHA to capture essential relationships between network
       flows.
12:    Step 3: Optimization and training
13:    Compute adaptive loss:
14:     $\mathcal{L}_{total} = \mathcal{L}_{CE} + \lambda_1 \mathcal{L}_{L2} + \lambda_2 \mathcal{L}_{contrastive}$ 
15:    where  $\mathcal{L}_{CE}$  is cross-entropy loss,  $\mathcal{L}_{L2}$  is L2 regularization, and
        $\mathcal{L}_{contrastive}$  is contrastive loss.
16:    Optimize hyperparameters using AWDV-based PSO-AWDV.
17:    Update model parameters  $\theta$  using gradient descent:
18:     $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{total}$ 
19:    Step 4: Model evaluation and validation
20:    Evaluate model performance on the validation set.
21:    Adjust training strategies based on validation results.
22:  end for
23:  if convergence criteria met then
24:    Stop training.
25:  end if
26: end for
27: Output: Save trained model parameters  $\theta^*$  for malicious traffic detection
    in real-world applications.
```

---

### 3.8.2. Adaptive loss function

The proposed model employs an adaptive loss function to dynamically adjust the learning process based on the characteristics of the data and the training stage. This is particularly important for handling imbalanced datasets, where malicious traffic samples are often

significantly outnumbered by benign traffic. The adaptive loss function integrates multiple components to achieve robust and accurate detection:

- **Cross-Entropy Loss:** The primary classification loss, which measures the discrepancy between predicted probabilities and true labels. It is formulated as:

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N y_i \log(\hat{y}_i) \quad (21)$$

where  $y_i$  denotes the true label,  $\hat{y}_i$  is the predicted probability, and  $N$  is the number of samples.

- **Dynamic Thresholding:** To address class imbalance, the adaptive loss incorporates dynamic thresholds that evolve according to data distribution. This prevents bias toward the majority class (benign traffic) and maintains sensitivity to the minority class (malicious traffic).
- **Contrastive Loss:** Improves feature separability by pulling embeddings of positive pairs closer while pushing negative pairs farther apart. This enhances the discriminative power of the learned feature space.
- **L2 Regularization:** Penalizes large weights to prevent overfitting and improve generalization.

The combined adaptive loss function is defined as:

$$\mathcal{L}_{total} = \mathcal{L}_{CE} + \lambda_1 \mathcal{L}_{L2} + \lambda_2 \mathcal{L}_{contrastive} \quad (22)$$

where  $\lambda_1$  and  $\lambda_2$  are weighting coefficients that balance the contributions of the regularization and contrastive components.

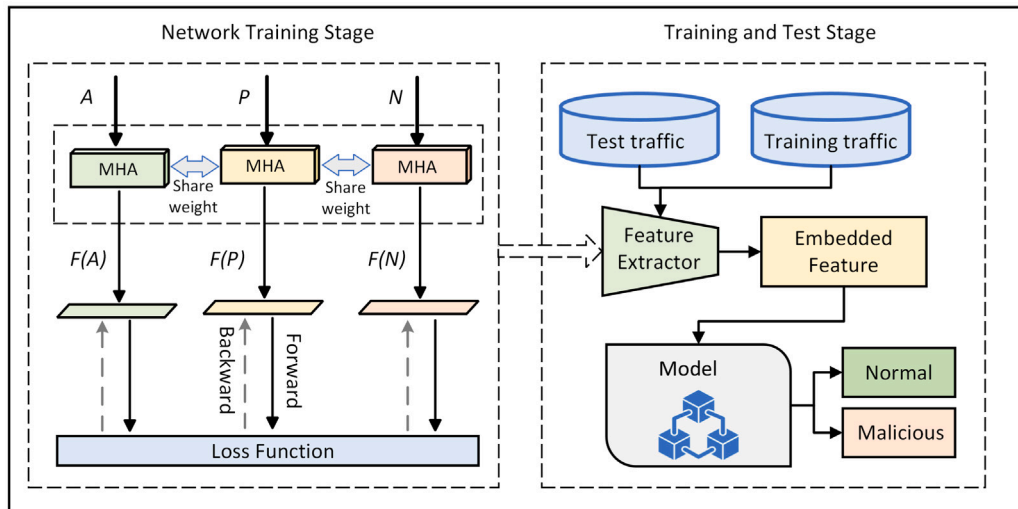
The adaptive loss refers to the dynamically adjusted classification loss (cross-entropy with thresholding) that accounts for class imbalance during training. In contrast, the combined adaptive loss integrates this adaptive loss with additional components — L2 regularization and contrastive loss — under a weighted formulation (Eq. (22)). This integration ensures both stability and discriminative feature learning, leading to faster and more robust convergence.

In our experiments, the weighting coefficients were empirically determined through grid search on the validation set. Specifically, we set  $\lambda_1 = 1 \times 10^{-4}$  to regularize the model without hindering convergence, and  $\lambda_2 = 0.2$  to balance the influence of the contrastive loss relative to the cross-entropy term. This configuration provided the best trade-off between generalization and stability across all datasets.

The adaptive loss function addresses class imbalance by incorporating dynamic thresholding, which modifies the decision boundary between benign and malicious classes during training. In highly imbalanced intrusion detection datasets, a fixed threshold (e.g., 0.5) often biases the classifier toward the majority class (benign traffic), resulting in a high false negative rate for the minority class (malicious traffic). To counter this, our framework updates the classification threshold  $\tau$  at each epoch based on the observed error rates on the validation set. Specifically, the threshold is shifted upward or downward depending on the relative values of the false negative rate (FNR) and false positive rate (FPR). This adaptive mechanism ensures that when the model tends to miss malicious samples (high FNR), the decision boundary becomes more sensitive to minority class instances, while still controlling false positives. The adjustment is guided by the AWDV optimization process, which evaluates the validation loss and updates the threshold to minimize false negatives for malicious traffic while controlling false positives for benign traffic. Formally, the threshold  $\tau$  at epoch  $t$  is updated as:

$$\tau_t = \tau_{t-1} + \eta \cdot (FNR_t - FPR_t) \quad (23)$$

where  $\eta$  is the learning rate for threshold adaptation, and  $FNR_t$  and  $FPR_t$  represent the false negative and false positive rates observed on the validation set at epoch  $t$ . This mechanism allows the decision boundary to evolve during training in response to class imbalance,



**Fig. 3.** Illustrates the two-phase detection framework for malicious network traffic. In the training phase (a), the model with multi-head attention modules is trained using shared weights and an adaptive loss function to optimize feature representations. In the testing phase (b), the feature extractor processes both training and test traffic to generate embedded features, which are then classified by the model into normal or malicious categories.

preventing the model from being overly conservative in detecting malicious traffic. By combining adaptive thresholding with cross-entropy and contrastive loss, the proposed framework achieves balanced sensitivity and specificity, thereby improving the detection of minority class attacks without sacrificing overall accuracy.

**Algorithm 2** Sampling procedure for contrastive learning in MNTD

```

1: Input: Traffic dataset  $D$  with labeled flows, batch size  $B$ , temporal window  $\Delta t$ 
2: for each batch do
3:   Randomly select  $B$  anchor flows  $\{x_i\}$  from  $D$ 
4:   for each anchor flow  $x_i$  do
5:     Select a positive sample  $x_j$  from the same class as  $x_i$ 
6:     ▷ Ensure temporal proximity:  $|t_i - t_j| \leq \Delta t$ 
7:     Select a negative sample  $x_k$  from a different class than  $x_i$ 
8:     Form triplet  $(x_i, x_j, x_k)$ 
9:   end for
10:  Construct batch of  $B$  triplets for contrastive loss computation
11: end for
12: Output: Batch of triplets (anchor, positive, negative) for contrastive learning

```

### 3.9. Data preprocessing

Network traffic is typically represented as a collection of flows, which serve as the fundamental detection granularity in many existing approaches (Fang et al., 2021). However, a single flow often provides limited information and may not fully capture the interactions between communication nodes. To address this limitation, we aggregate temporally correlated flows into communication channels. A communication channel is defined as a sequence of flows sharing the same source IP, destination IP, and destination port within a specific time window. This allows the model to capture temporal dependencies and interactions, improving the detection of malicious activities.

All experiments in this study were conducted using the publicly available CSV versions of the datasets (CICIDS2017, CIRA-CIC-DoHBrw2020, BoT-IoT, and UNSW-NB15), rather than raw PCAP files. These CSVs contain flow-level records pre-extracted using CICFlowMeter or equivalent tools, ensuring reproducibility and comparability with prior work. Our GitHub repository has been updated to explicitly reflect this setup.

The preprocessing steps included: (i) removal of incomplete flows with missing values, (ii) normalization of numerical attributes using Min-Max scaling to  $[0,1]$ , and (iii) one-hot encoding of categorical attributes such as protocol and service type. No manual feature engineering was applied beyond these preprocessing steps.

The extracted features fall into three categories:

- **Statistical features:** flow-level statistics such as duration, total packets, total bytes, average packet size, flow rate, and inter-arrival time. These describe overall traffic behavior.
- **Payload-based features:** aggregated characteristics of the payloads in each flow, e.g., minimum/maximum payload length, mean payload size, and byte distribution. For datasets where raw payloads were not available (e.g., UNSW-NB15, CIC-DoHBrw2020), only flow-level features were used.
- **Non-statistical features:** categorical and protocol-level attributes, such as protocol type, connection state, TCP flags, and service. These provide metadata that complements statistical features.

Mathematically, statistical features are represented as:

$$x_{\text{stat}} = [S_1, S_2, \dots, S_{N_s}] \quad (24)$$

Payload features are represented as:

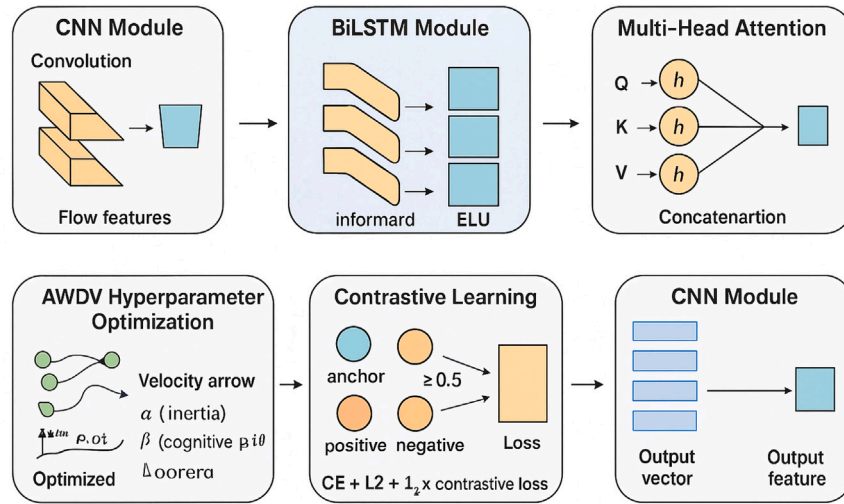
$$x_{\text{payload}} = [P_1, P_2, \dots, P_{N_p}], \quad P_i = [b_{i1}, b_{i2}, \dots, b_{iM}], \quad b_{ij} \in [0, 255] \quad (25)$$

where  $x_{\text{stat}}$  represents the statistical features,  $x_{\text{payload}}$  the payload features,  $P_i$  the  $i$ th packet, and  $b_{ij}$  the  $j$ th byte of the  $i$ th packet. Payload bytes are normalized to  $[0, 1]$  by dividing by 255. Statistical features are normalized using min-max scaling:

$$\bar{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (26)$$

To improve clarity and reproducibility, we provide a flowchart in Fig. 5, which illustrates the complete preprocessing pipeline from raw traffic capture to feature matrix construction. The diagram outlines how individual packets are grouped into flows, features are extracted and normalized, and communication channels are constructed to represent temporal correlations. This process ensures consistent and structured input to the MNTD model.

Finally, flows are aggregated into communication channels, which are then passed to the feature extraction network. This pipeline ensures that the model leverages both local and temporal dependencies, enhancing malicious traffic detection.



**Fig. 4.** Illustration of the key submodules in the proposed MNTD framework: (a) CNN module for extracting local spatial features from flow inputs, (b) BiLSTM module with ELU activation to capture bidirectional temporal dependencies, (c) Multi-head attention mechanism to emphasize the most relevant sequence features, (d) AWDV-based hyperparameter optimization process, (e) contrastive learning strategy using anchor, positive, and negative samples, and (f) the final feature aggregation and output stage. These visualizations provide an intuitive understanding of how each component contributes to robust malicious traffic detection.

### 3.9.1. Handling class imbalance

Intrusion detection datasets often exhibit significant class imbalance, where benign traffic vastly exceeds malicious instances. To address this challenge, we adopted a combined strategy involving both data-level and algorithm-level solutions. At the data level, we employed the Synthetic Minority Over-sampling Technique (SMOTE) to generate synthetic samples for the minority (malicious) class, thereby improving class balance during training.

At the algorithmic level, instead of using static class weighting, we designed an adaptive loss function tailored to imbalanced scenarios. This loss function integrates three key components: cross-entropy for baseline classification, dynamic thresholding to adjust decision boundaries based on evolving data distributions, and contrastive loss to improve feature separability between benign and malicious traffic. This formulation allows the model to better capture underrepresented patterns and maintain high sensitivity to attack instances without explicitly assigning static class weights.

Furthermore, the AWDV optimizer contributes to imbalance mitigation by enhancing the exploration–exploitation balance during optimization, which promotes robust generalization across both majority and minority classes.

This integrated strategy has proven effective in reducing false positives and false negatives, as evidenced by our experimental results. While SMOTE and the adaptive loss function performed well in this study, we acknowledge that other techniques — such as focal loss or sample reweighting — may offer additional benefits in more extreme imbalance scenarios, and we plan to explore such approaches in future work.

## 4. Experimental setup

### 4.1. Description of dataset

- The CICIDS2017 (Sharafaldin et al., 2018) dataset, developed by the Canadian Institute for Cybersecurity, serves as a comprehensive benchmark for network intrusion detection systems. It features a realistic representation of network traffic, including both benign and various types of malicious activities such as DoS, DDoS, Web attacks, Infiltration, Patator, Heartbleed, Portscan, and Botnet attacks. The dataset was collected in a controlled environment simulating a typical small enterprise network, using

tools such as Argus and Bro-IDS (now Zeek) for traffic generation and capture. It contains a total of approximately 12 million benign packets and 4.5 million attack packets, with a breakdown of 2,273,097 benign flows and 231,073 attack flows. The dataset provides a rich set of features extracted from network flows, including metrics like packet size, flow duration, and inter-arrival times, making it a valuable resource for developing and benchmarking intrusion detection methodologies.

- The CIRA-CIC-DoHBrw2020 (MontazeriShatoori et al., 2020) dataset focuses on DNS over HTTPS (DoH) traffic and includes a diverse array of traffic patterns. It categorizes the data into three groups: non-DoH, benign-DoH, and malicious-DoH. The benign-DoH data represents normal DoH traffic generated by browsers such as Google Chrome and Mozilla Firefox accessing the Alexa top 10,000 domains, while the malicious-DoH traffic includes covert channels created using tools like iodine and dnscat2. Non-DoH consists of regular HTTPS traffic unrelated to DNS tunneling. The dataset contains significant packet distributions, including 5,609,000 packets from Chrome and 4,943,000 from Firefox, providing a robust benchmark for evaluating intrusion detection systems in encrypted traffic environments.
- The BoT-IoT (Koroniotis et al., 2019) dataset was developed to address the increasing challenges of detecting botnet-driven attacks in IoT environments. It includes a wide range of attack categories such as DDoS, DoS, reconnaissance, and information theft, alongside benign IoT traffic. The traffic was generated in a realistic testbed environment using virtual machines and IoT devices, ensuring diverse attack scenarios. The dataset contains over 70 million records with labeled flows, providing both packet-level and flow-level features that make it highly suitable for evaluating intrusion detection techniques in resource-constrained IoT settings.
- The UNSW-NB15 (Moustafa and Slay, 2015) dataset, created at the Australian Centre for Cyber Security (ACCS), is another widely used benchmark that includes modern and diverse attack types. It was generated using the IXIA PerfectStorm tool and captures realistic hybrid traffic, combining normal activities with nine categories of attacks such as Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. The dataset contains around 2.5 million records with 49 extracted features, including both flow-based and content-based attributes.

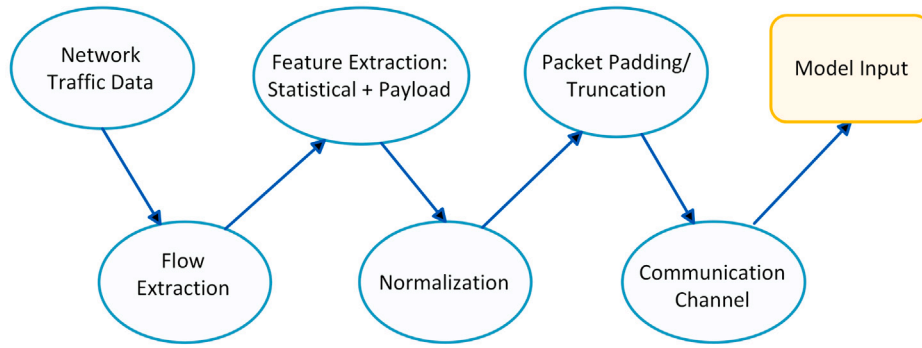


Fig. 5. Flowchart of the preprocessing pipeline for constructing communication channels and preparing features for the MNTD model.

Table 2

Dataset splitting for training, validation, and testing.

| Dataset        | Total flows | Training (80%) | Validation (10%) | Testing (10%) |
|----------------|-------------|----------------|------------------|---------------|
| CICIDS2017     | 2,504,170   | 2,003,336      | 250,417          | 250,417       |
| CIC-DoHBrw2020 | 10,552,000  | 8,441,600      | 1,055,200        | 1,055,200     |
| BoT-IoT        | 72,000,000  | 57,600,000     | 7,200,000        | 7,200,000     |
| UNSW-NB15      | 2,540,044   | 2,032,035      | 254,004          | 254,005       |

Table 3

Dataset details: benign/malicious flows and number of features.

| Dataset             | Benign flows | Malicious flows | No. of features |
|---------------------|--------------|-----------------|-----------------|
| CICIDS2017          | 2,273,097    | 231,073         | 78              |
| CIRA-CIC-DoHBrw2020 | 8,091,000    | 2,461,000       | 43              |
| BoT-IoT             | 477,677      | 3,668,045       | 42              |
| UNSW-NB15           | 2,218,761    | 321,283         | 49              |

Its balanced design and wide coverage of attack categories make it a valuable complement to CICIDS2017 and BoT-IoT for evaluating the generalization of intrusion detection models across different network environments.

To ensure reproducibility and consistent evaluation, all datasets were split into training, validation, and testing sets using an 80:10:10 ratio. The splits were stratified to preserve the class distribution of benign and malicious traffic in all subsets. Table 2 summarizes the flow-level distribution used in our experiments.

Table 3 summarizes additional details of the datasets used in this work. Specifically, we provide the number of benign and malicious flows in each dataset, along with the number of extracted features after preprocessing. The CICIDS2017 dataset contains 2,273,097 benign flows and 231,073 malicious flows, with 78 features used. The CIRA-CIC-DoHBrw2020 dataset comprises 8,091,000 benign flows and 2,461,000 malicious flows, with 43 features extracted. The BoT-IoT dataset includes 477,677 benign flows and 3,668,045 malicious flows, represented by 42 features. Finally, the UNSW-NB15 dataset contains 2,218,761 benign flows and 321,283 malicious flows, with 49 features available. This information provides a clear view of the class distribution and feature dimensionality employed in our evaluation.

The selection of datasets was guided by the need to evaluate the proposed framework under diverse and realistic attack scenarios. CICIDS2017 was chosen as it is a comprehensive benchmark containing a wide variety of traditional intrusions (e.g., DDoS, DoS, infiltration, botnets, port scanning), making it highly representative for general-purpose intrusion detection tasks. In contrast, the CIRA-CIC-DoHBrw2020 dataset was included due to the increasing adoption of encrypted communication protocols such as DNS-over-HTTPS (DoH), which pose significant challenges to intrusion detection systems because payload visibility is limited. By including this dataset, our evaluation explicitly addresses the problem of detecting malicious traffic

in encrypted settings, a scenario highly relevant to modern networks. To further strengthen generalizability, we additionally incorporated two complementary datasets: UNSW-NB15, which provides a modern replacement to KDD-style datasets with realistic background traffic and diverse contemporary attack types, and BoT-IoT, which specifically targets IoT ecosystems with large-scale botnet and DDoS traffic. Together, these four datasets ensure that the evaluation spans traditional enterprise networks, encrypted traffic, and IoT environments, thereby demonstrating that the proposed MNTD framework is adaptable and effective across a wide spectrum of real-world intrusion detection scenarios.

#### 4.2. Experimental setup

All experiments were conducted on a workstation running Ubuntu 20.04 LTS with an NVIDIA RTX 3090 GPU (24 GB memory), 128 GB RAM, and an AMD Ryzen Threadripper 3960X CPU (3.8 GHz, 24 cores).

The implementation was developed in Python 3.9.13. Deep learning components were built using TensorFlow 2.10 and Keras 2.10, while Scikit-learn 1.1.3 was used for preprocessing and evaluation utilities. NumPy 1.23.5 and Pandas 1.5.2 were employed for numerical and tabular operations. Matplotlib 3.6.2 and Seaborn 0.12.1 were used for visualization. CUDA 11.7 and cuDNN 8.5 were installed to enable GPU acceleration.

To ensure reproducibility, random seeds were fixed across TensorFlow, NumPy, and Python's random module. The code and configuration files, including preprocessing scripts and hyperparameter settings.

##### 4.2.1. Time complexity analysis

The computational complexity of each component in the proposed MNTD framework is summarized in Table 4. This analysis provides insights into the efficiency of the model with respect to input size, sequence length, feature dimension, and number of attention heads.

#### 4.3. Hyperparameter adjustment

The selection of hyperparameters is crucial for effective model training, as inappropriate choices can lead to overly complex networks and increased computation time, as shown in Table 5. For the proposed MNTD framework, key hyperparameters include those related to the CNN, BiLSTM, MHA, and the adaptive loss function. To identify robust

**Table 4**

Time complexity analysis of MNTD components.

| Component                   | Time complexity                                  |
|-----------------------------|--------------------------------------------------|
| Convolutional layer (CNN)   | $\mathcal{O}(n \cdot k \cdot d)$                 |
| BiLSTM layer                | $\mathcal{O}(n \cdot d^2)$                       |
| Multi-head attention        | $\mathcal{O}(h \cdot n^2 \cdot d)$               |
| Contrastive learning module | $\mathcal{O}(n \cdot d)$                         |
| AWDV optimization           | $\mathcal{O}(i \cdot p \cdot d)$                 |
| <b>Overall complexity</b>   | $\mathcal{O}(n \cdot d^2 + h \cdot n^2 \cdot d)$ |

settings, we performed grid search within predefined ranges for each parameter.

The final configuration, which yielded the best validation performance across all datasets, was: a Conv1D layer with 64 filters (kernel size = 3), max pooling size of 2, BiLSTM layers with 128 units each, and an MHA module with 4 heads. The dense layer had 64 neurons, followed by a dropout rate of 0.05. We adopted a learning rate of  $2e-3$ , batch size of 128,  $\lambda_1 = 1 \times 10^{-4}$  for L2 regularization, and  $\lambda_2 = 0.2$  for the contrastive loss term. The model was trained for 50 epochs, stabilizing by approximately epochs 20–30 across all compared baselines.

#### 4.4. Comparison with baseline methods

Several recent studies have explored hybrid CNN-BiLSTM architectures enhanced with attention mechanisms to improve the performance of network traffic detection systems. For instance, the GWO-CNN-BiLSTM model (Yu et al., 2024a) employs Grey Wolf Optimization to fine-tune model parameters for enhanced intrusion detection. Similarly, Wang (2024) proposed a WGAN-based CNN-BiLSTM model incorporating attention mechanisms within a cloud-edge computing framework to address the challenges posed by data imbalance. Zhang et al. (2023b) introduced a BiLSTM-based architecture equipped with a multi-head attention mechanism to effectively capture long-range temporal dependencies in network flows. The AB-BiLSTM model (Ren et al., 2019) improves conventional recurrent architectures by leveraging attention mechanisms to model long-distance dependencies. Additionally, CNN-LSTM (Sun et al., 2020b), DNN (Ahmad et al., 2022) and DCNN-BiLSTM (Hnamte and Hussain, 2023) architectures combine convolutional and recurrent layers to extract both spatial and sequential features, thereby improving detection accuracy and robustness. These models have shown promising results and are typically validated through repeated experiments to ensure consistency and reliability.

In contrast, the proposed MNTD framework not only incorporates CNN, BiLSTM, and multi-head attention modules but also introduces several advanced components, including contrastive learning, AWDV optimization, and an adaptive loss function with integrated regularization. These enhancements enable MNTD to learn more discriminative and transferable feature representations, improving its ability to detect zero-day attacks and generalize across diverse network environments. In addition, the inclusion of communication-channel level modeling and dynamic thresholding significantly enhances the model's detection sensitivity and computational efficiency.

For fair comparison, all baseline models were re-implemented in our environment to ensure consistency across preprocessing, training, and evaluation. Where available, we referred to the original authors' published code and configurations. In cases where complete implementations were not publicly available, the models were reproduced from scratch strictly following the descriptions provided in the corresponding papers. The following measures were taken to maintain fairness: (i) all models were trained on the same datasets with identical train/validation/test splits; (ii) preprocessing steps such as normalization and encoding were uniformly applied; (iii) all models were trained for 50 epochs with early stopping based on validation loss; (iv) the Adam optimizer and batch size of 128 were used unless explicitly defined

otherwise in the original paper; and (v) learning rates were tuned within the same search space as the proposed model to prevent bias.

No significant deviations were introduced from the original architectures of the baseline models beyond ensuring compatibility with our experimental pipeline (e.g., adapting input dimensions or replacing unavailable layers with equivalent standard implementations). These clarifications ensure that performance comparisons reflect genuine differences in model capability rather than inconsistencies in implementation or training conditions.

##### 4.4.1. Baseline architectures and complexity comparison

To provide a clearer context for the performance gains achieved by the proposed MNTD framework, we detail the architectural components and complexities of the baseline models used in our evaluation.

All models were trained for a maximum of 50 epochs with early stopping (patience = 5). In practice, most runs converged before the 30th epoch, as indicated by stable validation loss. Early stopping prevented unnecessary training and mitigated overfitting while preserving the best-performing model weights. The learning rates were adjusted on the basis of each model's convergence characteristics. The same input features were used for all models.

Table 6 compares the models in terms of architecture depth and trainable parameters. As shown, MNTD achieves higher performance with a moderate parameter size and a more sophisticated attention-based design, allowing better generalization while maintaining computational efficiency.

#### 4.5. Experiment evaluation design

We employ classification evaluation metrics such as accuracy, precision, recall, F1-score, FPR (False Positive Rate), FNR (False Negative Rate), and Receiver Operating Characteristic (ROC) curve to assess the effectiveness of the proposed model. In this context, malicious network traffic is considered positive, while benign traffic is treated as negative. The following metrics are defined:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (27)$$

$$Precision = \frac{TP}{TP + FP} \quad (28)$$

$$Recall = \frac{TP}{TP + FN} \quad (29)$$

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (30)$$

$$FPR = \frac{FP}{FP + TN} \quad (31)$$

$$FNR = \frac{FN}{FN + TP} \quad (32)$$

##### 4.5.1. Cosine similarity threshold for anomaly detection

Cosine similarity measures the similarity between two traffic flow embeddings, calculated as:

$$\text{cosine\_similarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \quad (33)$$

A flow is classified as normal or anomalous based on a predefined threshold  $\tau$ :

$$\text{Label} = \begin{cases} \text{Normal} & \text{if cosine\_similarity} \geq \tau \\ \text{Anomalous} & \text{if cosine\_similarity} < \tau \end{cases} \quad (34)$$

In our experiments, we empirically set the cosine similarity threshold to  $\tau = 0.5$ , which provides a balanced trade-off between true positive and false positive rates (see Table 7).

The selected threshold  $\tau = 0.5$  is highlighted in bold and represents the optimal trade-off between detection accuracy and false alarms.

**Table 5**  
Final optimized architecture and hyperparameters of the proposed MNTD framework.

| Layer                | Type                      | Output dimension | Description (Final settings)                 |
|----------------------|---------------------------|------------------|----------------------------------------------|
| Input layer          | Raw flow features         | (Batch, 100)     | Normalized network traffic features          |
| Conv1D               | 1D Convolution + ReLU     | (Batch, 64, 50)  | 64 filters, kernel size = 3, stride = 1      |
| MaxPooling1D         | Pooling                   | (Batch, 64, 25)  | Pool size = 2                                |
| BiLSTM layer 1       | Bidirectional LSTM + ELU  | (Batch, 128)     | 64 units $\times$ 2 directions               |
| BiLSTM layer 2       | Bidirectional LSTM + ELU  | (Batch, 128)     | Deeper temporal modeling                     |
| Multi-head attention | Attention mechanism       | (Batch, 128)     | 4 attention heads                            |
| Dense layer 1        | Fully Connected + ReLU    | (Batch, 64)      | Feature compression                          |
| Dropout layer        | Regularization            | (Batch, 64)      | Dropout rate = 0.05                          |
| Dense layer 2        | Fully Connected + Softmax | (Batch, 2)       | Binary classification (benign vs. malicious) |

**Table 6**  
Architectural complexity and design comparison of baseline models and the proposed MNTD.

| Model          | Layers   | Trainable parameters | Key components                   |
|----------------|----------|----------------------|----------------------------------|
| DNN            | 3        | 85K                  | Dense, ReLU, Dropout             |
| GWO-CNN-BiLSTM | 4        | 100K                 | Conv1D, BiLSTM, GWO Optimization |
| WGAN           | 3        | 250K                 | Generator, Discriminator, LSTM   |
| AB-BiLSTM      | 4        | 320K                 | BiLSTM, Attention, Dense         |
| CNN-LSTM       | 5        | 410K                 | Conv1D, LSTM, Dropout            |
| DCNN-BiLSTM    | 6        | 480K                 | Deep CNN, BiLSTM, Dropout        |
| <b>MNTD</b>    | <b>7</b> | <b>520K</b>          | CNN, BiLSTM, MHA, AWDV, CL       |

**Table 7**  
Effect of cosine similarity threshold on detection performance.

| Threshold ( $\epsilon$ ) | True Positive Rate (TPR) | False Positive Rate (FPR) |
|--------------------------|--------------------------|---------------------------|
| 0.2                      | 92.3%                    | 8.5%                      |
| 0.3                      | 93.4%                    | 7.8%                      |
| 0.4                      | 94.2%                    | 7.2%                      |
| <b>0.5</b>               | <b>96.8%</b>             | <b>5.6%</b>               |
| 0.6                      | 93.5%                    | 6.5%                      |
| 0.7                      | 94.6%                    | 5.9%                      |
| 0.8                      | 94.8%                    | 7.5%                      |

## 5. Experimental results and analysis

In this section, we present the results of our experimental evaluation and analyze the performance of each model.

### 5.1. Comparative evaluation of MNTD against baseline approaches

In addition to conventional baselines, we compared the proposed MNTD framework with several advanced hybrid architectures, including DNN, GWO-CNN-BiLSTM, WGAN-CNN-BiLSTM, CNN-LSTM, AB-BiLSTM, and DCNN-BiLSTM, which have been extensively used in recent intrusion detection studies. While these models demonstrate strong capabilities for capturing spatial or temporal patterns, many rely on fixed attention mechanisms or shallow optimization techniques. In contrast, MNTD incorporates MHA to dynamically attend to critical time steps across network flows, enabling an improved focus on relevant temporal patterns. Moreover, the integration of contrastive learning and AWDV-based hyperparameter optimization improves the model's ability to generalize to unseen attack scenarios and fine-tune its decision boundaries effectively.

Table 8 presents the comparative results across four benchmark datasets: CICIDS2017, CIRA-CIC-DoHBrw2020, UNSW-NB15, and BoT-IoT. On CICIDS2017, MNTD achieved the highest performance across all evaluation metrics—accuracy of 98.52%, precision of 97.99%, recall of 98.43%, and an F1-score of 98.98%. These results represent a significant improvement over other approaches; for instance, GWO-CNN-BiLSTM and WGAN-CNN-BiLSTM achieved accuracies of 89.98% and 85.53%, respectively, while AB-BiLSTM attained a competitive accuracy of 91.18%.

Similarly, on the CIRA-CIC-DoHBrw2020 dataset, MNTD demonstrated superior detection performance with an accuracy of 98.82%, precision of 98.79%, recall of 98.53%, and an F1-score of 98.66%. By comparison, GWO-CNN-BiLSTM and WGAN-CNN-BiLSTM obtained accuracies of 96.07% and 90.21%, respectively. The consistent and superior performance of MNTD in both datasets underscores its effectiveness in diverse network environments.

The same trend was observed on the UNSW-NB15 and BoT-IoT datasets. On UNSW-NB15, MNTD achieved 97.91% accuracy, outperforming the strongest baseline (DCNN-BiLSTM at 92.37%). For BoT-IoT, MNTD reached 98.65% accuracy, again surpassing DCNN-BiLSTM (94.20%). These findings confirm that MNTD consistently generalizes well across heterogeneous datasets, maintaining high precision and recall while minimizing false positives and false negatives.

Overall, the results in Table 8 indicate that MNTD exhibits greater robustness, particularly under noisy and obfuscated traffic conditions. For example, although DCNN-BiLSTM performs well in general, it tends to suffer from higher false positive rates and slower convergence due to its deeper architecture and limited adaptivity. In contrast, MNTD's streamlined yet modular architecture ensures reliable detection and efficient training, offering both practical advantages and methodological novelty over existing state-of-the-art models.

It is worth noting that in Table 8, the reported values appear nearly identical across the datasets. This similarity is not the result of a multiclass weighted averaging scheme (as our task is strictly binary classification), but rather reflects the class distribution and error profile of the proposed framework. Because the datasets were preprocessed using stratified splits that preserve class balance, the numbers of benign and malicious flows in the test sets are comparable. In addition, the false positive and false negative counts produced by MNTD are both extremely low, such that correctly classified samples dominate the evaluation metrics. Under these conditions, overall accuracy tends to converge toward recall, since almost all malicious samples are correctly detected while class proportions remain balanced. This observation underscores the framework's effectiveness in minimizing both type I and type II errors simultaneously.

All experiments in this study were conducted under a binary classification setting, where traffic flows were labeled as either benign or malicious. This setup ensures consistent evaluation across all four datasets and reflects the practical objective of real-time malicious traffic detection.

#### 5.1.1. Inference latency analysis

Table 9 compares the inference latency and throughput of the proposed MNTD with baseline models, including DNN, GWO-CNN-BiLSTM, WGAN-CNN-BiLSTM, CNN-LSTM, AB-BiLSTM, and DCNN-BiLSTM. The results clearly indicate that traditional architectures such as DNN and hybrid CNN-BiLSTM variants exhibit higher latency, with inference times ranging between 2–4 ms per flow and relatively low throughput. Among the baselines, DCNN-BiLSTM achieves the best performance, with a latency of 1.85 ms/flow and throughput of 540 flows/s, but still falls short of real-time deployment requirements in high-speed networks. In contrast, the proposed MNTD framework achieves sub-millisecond inference latency (0.93 ms/flow) and a throughput of 1075

**Table 8**

Performance comparison of the proposed MNTD and baseline models on four benchmark datasets.

| Dataset             | Model                | Accuracy (%) | Precision (%) | Recall (%)   | F1-score (%) |
|---------------------|----------------------|--------------|---------------|--------------|--------------|
| CICIDS2017          | DNN                  | 91.23        | 89.97         | 90.34        | 90.10        |
|                     | GWO-CNN-BiLSTM       | 89.98        | 90.85         | 91.89        | 90.19        |
|                     | WGAN                 | 85.53        | 85.56         | 87.53        | 87.21        |
|                     | AB-BiLSTM            | 91.18        | 90.92         | 91.18        | 90.62        |
|                     | CNN-LSTM             | 89.86        | 91.18         | 88.66        | 89.53        |
|                     | DCNN-BiLSTM          | 94.29        | 92.34         | 92.68        | 93.17        |
|                     | <b>Proposed MNTD</b> | <b>98.52</b> | <b>97.99</b>  | <b>98.43</b> | <b>98.98</b> |
| CIRA-CIC-DoHBrw2020 | DNN                  | 89.74        | 88.25         | 88.91        | 88.58        |
|                     | GWO-CNN-BiLSTM       | 96.07        | 96.60         | 96.07        | 95.42        |
|                     | WGAN                 | 90.21        | 88.86         | 90.20        | 89.86        |
|                     | AB-BiLSTM            | 91.56        | 90.31         | 91.56        | 90.14        |
|                     | CNN-LSTM             | 93.20        | 86.86         | 93.20        | 89.91        |
|                     | DCNN-BiLSTM          | 93.36        | 91.48         | 93.36        | 91.70        |
|                     | <b>Proposed MNTD</b> | <b>98.82</b> | <b>98.79</b>  | <b>98.53</b> | <b>98.66</b> |
| UNSW-NB15           | DNN                  | 88.67        | 87.45         | 86.92        | 87.18        |
|                     | GWO-CNN-BiLSTM       | 90.52        | 91.13         | 90.25        | 90.52        |
|                     | WGAN                 | 87.20        | 85.91         | 86.75        | 86.33        |
|                     | AB-BiLSTM            | 91.06        | 90.12         | 90.83        | 90.47        |
|                     | CNN-LSTM             | 90.15        | 89.37         | 89.88        | 89.62        |
|                     | DCNN-BiLSTM          | 92.37        | 91.56         | 91.92        | 91.74        |
|                     | <b>Proposed MNTD</b> | <b>97.91</b> | <b>97.42</b>  | <b>97.80</b> | <b>97.61</b> |
| BoT-IoT             | DNN                  | 90.12        | 89.56         | 89.72        | 89.64        |
|                     | GWO-CNN-BiLSTM       | 92.83        | 92.46         | 92.91        | 92.68        |
|                     | WGAN                 | 89.41        | 88.72         | 89.15        | 88.93        |
|                     | AB-BiLSTM            | 93.06        | 92.74         | 92.88        | 92.81        |
|                     | CNN-LSTM             | 92.14        | 91.23         | 91.84        | 91.53        |
|                     | DCNN-BiLSTM          | 94.20        | 93.58         | 94.01        | 93.79        |
|                     | <b>Proposed MNTD</b> | <b>98.65</b> | <b>98.21</b>  | <b>98.60</b> | <b>98.40</b> |

**Table 9**

Inference latency and throughput comparison between the proposed MNTD and baseline models.

| Model                | Inference latency (ms/flow) | Throughput (flows/s) |
|----------------------|-----------------------------|----------------------|
| DNN                  | 4.25                        | 235                  |
| GWO-CNN-BiLSTM       | 3.70                        | 270                  |
| WGAN-CNN-BiLSTM      | 3.45                        | 290                  |
| CNN-LSTM             | 2.95                        | 340                  |
| AB-BiLSTM            | 2.10                        | 475                  |
| DCNN-BiLSTM          | 1.85                        | 540                  |
| <b>Proposed MNTD</b> | <b>0.93</b>                 | <b>1075</b>          |

flows/s, significantly outperforming all baselines. This efficiency gain is attributed to the integration of multi-head attention and AWDV-based optimization, which streamline feature extraction and hyperparameter tuning, thereby reducing computational overhead. These results highlight MNTD's suitability for real-time intrusion detection applications, especially in latency-sensitive environments such as industrial networks and Software-Defined Networking (SDN) controllers.

### 5.2. Evaluating the efficacy of optimization techniques

To validate the effectiveness of the proposed AWDV optimization strategy, we conducted a comprehensive comparison against widely used hyperparameter optimization methods, including Grid Search (GS), Genetic Algorithm (GA), Random Search (RS), Particle Swarm Optimization (PSO), and Bayesian Optimization (BO). The performance across all four benchmark datasets (CIRA-CIC-DoHBrw2020, CICIDS2017, UNSW-NB15, and BoT-IoT) is summarized in Table 10 and visualized in Fig. 6.

The results indicate that AWDV consistently achieves superior performance in terms of accuracy, precision, recall, and F1-score, while maintaining very low false positive and false negative rates. Moreover, AWDV converges substantially faster than other methods, with average optimization times reduced by 25%–40% compared to GS and BO, and over 50% compared to GA. This efficiency is attained without compromising model stability, as AWDV dynamically adapts thresholds during the search process, effectively balancing exploration and exploitation.

For instance, on the CIRA-CIC-DoHBrw2020 dataset, AWDV achieved an accuracy of 98.82% with a convergence time of 99.85 s, compared to 94.31% accuracy and 186.44 s for GS, and 91.95% accuracy with 219.66 s for GA. Similar trends were observed across the other datasets, demonstrating that AWDV consistently provides the best trade-off between detection accuracy and computational efficiency.

These findings highlight that AWDV not only enhances the quality of hyperparameter selection but also improves the robustness of the proposed MNTD framework in highly imbalanced and complex network environments. The integration of AWDV is thus critical for enabling the framework to scale efficiently across diverse datasets and attack scenarios.

### 5.3. Sensitivity analysis of AWDV optimization

Table 11 summarizes the sensitivity analysis of the AWDV algorithm with respect to different initial hyperparameter settings on four benchmark datasets. This analysis evaluates how variations in learning rate, dropout ratio, and batch size affect model performance and convergence, highlighting AWDV's robustness and stability.

For the CICIDS2017 dataset, three configurations were tested. The model achieved an accuracy of 98.78% with a learning rate of 0.005, a dropout of 0.05, and a batch size of 128. A second configuration with a lower learning rate (0.002) and dropout (0.02) improved accuracy slightly to 98.52%, though it required more training time (210.52 s). The third configuration with the lowest learning rate (0.001) led to a further drop in accuracy to 96.62% and the longest convergence time (252.54 s). These results indicate that AWDV is sensitive to the extremes of the hyperparameter ranges, with mid-range values providing the best trade-off between accuracy and efficiency.

A similar trend was observed for the CIC-DoHBrw2020 dataset. The highest accuracy of 98.82% and the shortest convergence time (58.85 s) were achieved with the same optimal values as in CICIDS2017. Lowering the learning rate and dropout caused a slight decline in performance and increased convergence time, illustrating the importance of careful hyperparameter initialization.

For the newly added UNSW-NB15 dataset, AWDV achieved its best performance (97.91%) with a learning rate of 0.005, dropout of 0.05,

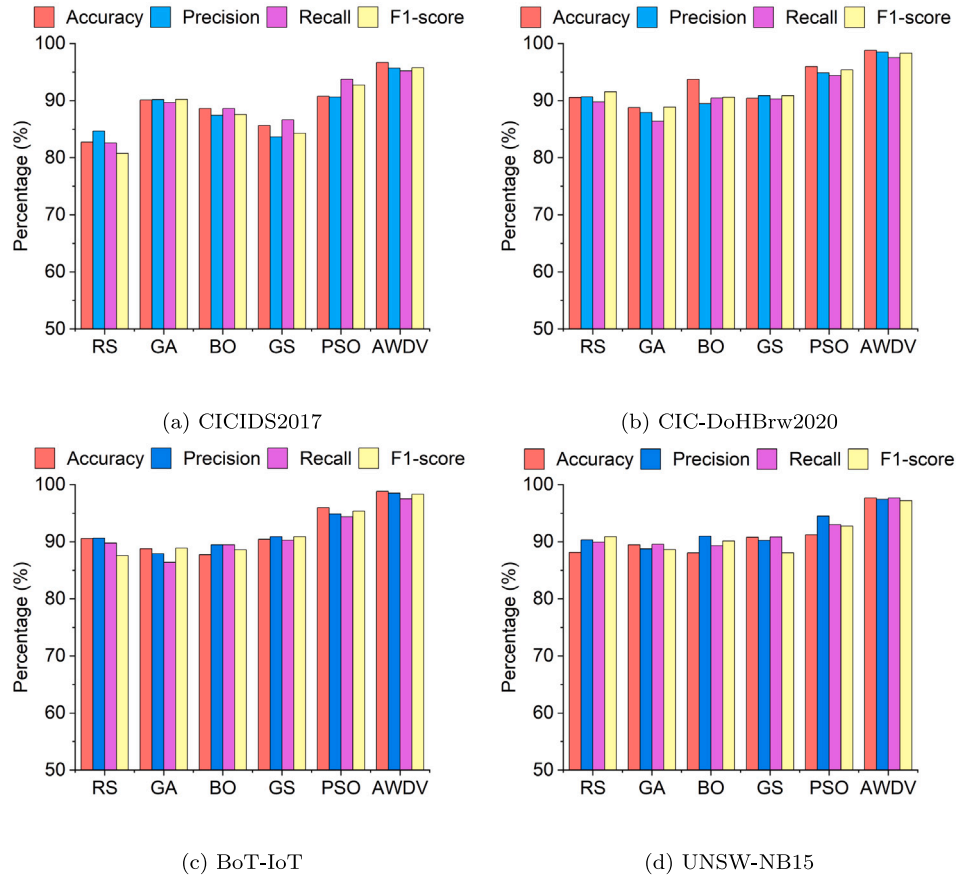


Fig. 6. Comparison of different optimization algorithms.

Table 10

Comparison of hyperparameter optimization techniques for the proposed MNTD framework (same model configuration).

| Dataset             | Optimizer | Accuracy (%) | Precision (%) | Recall (%)   | F1-score (%) | FPR           | FNR           | DR (%)       | Time (s)      |
|---------------------|-----------|--------------|---------------|--------------|--------------|---------------|---------------|--------------|---------------|
| CIRA-CIC-DoHBrw2020 | AWDV      | <b>98.82</b> | <b>98.79</b>  | <b>98.82</b> | <b>98.66</b> | <b>0.0014</b> | <b>0.1176</b> | <b>99.52</b> | <b>99.85</b>  |
|                     | GS        | 94.31        | 94.22         | 94.16        | 94.19        | 0.0021        | 0.2215        | 97.41        | 186.44        |
|                     | GA        | 91.95        | 91.41         | 89.88        | 89.66        | 0.0027        | 0.2442        | 96.93        | 219.66        |
|                     | RS        | 88.82        | 89.32         | 87.70        | 88.41        | 0.0029        | 0.2518        | 94.71        | 236.42        |
|                     | PSO       | 93.03        | 93.55         | 93.92        | 93.66        | 0.0026        | 0.2325        | 96.84        | 198.31        |
|                     | BO        | 90.26        | 90.83         | 90.15        | 90.00        | 0.0038        | 0.2164        | 96.16        | 157.56        |
| CICIDS2017          | AWDV      | <b>98.52</b> | <b>97.99</b>  | <b>98.43</b> | <b>98.98</b> | <b>0.0016</b> | <b>0.3027</b> | <b>98.89</b> | <b>192.61</b> |
|                     | GS        | 91.52        | 90.42         | 90.80        | 90.61        | 0.0055        | 0.3652        | 92.48        | 294.26        |
|                     | GA        | 94.06        | 93.95         | 93.22        | 93.11        | 0.0029        | 0.3885        | 96.97        | 231.42        |
|                     | RS        | 90.48        | 88.83         | 89.36        | 89.08        | 0.0036        | 0.4012        | 95.12        | 309.36        |
|                     | PSO       | 94.62        | 94.11         | 94.55        | 94.22        | 0.0028        | 0.3725        | 93.44        | 284.64        |
|                     | BO        | 89.13        | 88.32         | 89.71        | 89.52        | 0.0031        | 0.3541        | 94.62        | 346.56        |
| UNSW-NB15           | AWDV      | <b>97.91</b> | <b>97.42</b>  | <b>97.80</b> | <b>97.61</b> | <b>0.0025</b> | <b>0.3156</b> | <b>97.68</b> | <b>107.44</b> |
|                     | GS        | 93.12        | 94.78         | 93.04        | 93.89        | 0.0133        | 0.3442        | 94.02        | 117.65        |
|                     | GA        | 94.85        | 93.22         | 93.70        | 93.42        | 0.0236        | 0.3615        | 96.69        | 164.29        |
|                     | RS        | 85.73        | 84.15         | 84.59        | 84.32        | 0.0068        | 0.4724        | 96.58        | 292.74        |
|                     | PSO       | 93.94        | 93.34         | 93.81        | 93.51        | 0.0034        | 0.3583        | 96.82        | 195.11        |
|                     | BO        | 91.54        | 91.51         | 91.93        | 91.42        | 0.0149        | 0.3367        | 96.91        | 285.82        |
| BoT-IoT             | AWDV      | <b>98.65</b> | <b>98.21</b>  | <b>98.60</b> | <b>98.40</b> | <b>0.0018</b> | <b>0.1942</b> | <b>98.71</b> | <b>109.12</b> |
|                     | GS        | 92.02        | 92.63         | 92.88        | 92.75        | 0.0043        | 0.2384        | 91.88        | 288.31        |
|                     | GA        | 88.58        | 88.15         | 87.41        | 88.28        | 0.0029        | 0.3621        | 96.42        | 332.84        |
|                     | RS        | 93.44        | 93.98         | 93.33        | 93.15        | 0.0132        | 0.2769        | 94.32        | 211.52        |
|                     | PSO       | 92.09        | 92.12         | 92.15        | 92.07        | 0.0127        | 0.2511        | 96.56        | 194.77        |
|                     | BO        | 90.86        | 89.42         | 89.72        | 89.51        | 0.0021        | 0.2296        | 90.73        | 376.04        |

**Table 11**  
Sensitivity analysis of AWDV with respect to initial hyperparameter settings on four benchmark datasets.

| Dataset        | Config ID | Learning rate | Dropout | Batch size | Accuracy (%) | Time (s) |
|----------------|-----------|---------------|---------|------------|--------------|----------|
| CICIDS2017     | Config 1  | 0.005         | 0.05    | 128        | 98.78        | 192.61   |
|                | Config 2  | 0.002         | 0.02    | 64         | 98.52        | 210.52   |
|                | Config 3  | 0.001         | 0.01    | 32         | 96.62        | 252.54   |
| CIC-DoHBrw2020 | Config 1  | 0.005         | 0.05    | 128        | 98.82        | 58.85    |
|                | Config 2  | 0.002         | 0.02    | 64         | 98.81        | 85.18    |
|                | Config 3  | 0.001         | 0.01    | 32         | 98.41        | 136.66   |
| UNSW-NB15      | Config 1  | 0.005         | 0.05    | 128        | 97.91        | 89.44    |
|                | Config 2  | 0.002         | 0.02    | 64         | 97.42        | 105.12   |
|                | Config 3  | 0.001         | 0.01    | 32         | 96.87        | 134.22   |
| BoT-IoT        | Config 1  | 0.005         | 0.05    | 128        | 98.65        | 109.12   |
|                | Config 2  | 0.002         | 0.02    | 64         | 98.21        | 130.55   |
|                | Config 3  | 0.001         | 0.01    | 32         | 97.95        | 158.40   |

and batch size of 128, while the lower learning rate and batch size configurations led to slightly reduced accuracies of 97.42% and 96.87%, respectively, with increasing convergence times. Similarly, for the BoT-IoT dataset, the highest accuracy of 98.65% was obtained with the same optimal configuration, whereas smaller learning rates and batch sizes resulted in a modest decrease in accuracy and longer training times.

Overall, the sensitivity analysis demonstrates that AWDV maintains stable performance across a wide range of hyperparameter initializations, including variations in learning rate, dropout, and batch size. Although peak performance is typically observed at mid-range values (e.g., learning rate = 0.002–0.005, dropout = 0.02–0.05, batch size = 64–128), AWDV consistently avoids severe degradation under suboptimal configurations. This robustness confirms that AWDV is less sensitive to initialization boundaries compared to traditional optimization methods such as Grid Search or Random Search, enhancing its reliability for practical deployment across diverse network environments.

#### 5.4. Model learning analysis

The learning process, as illustrated in Table 12, demonstrates the effectiveness of the proposed MNTD framework across four benchmark datasets. This analysis evaluates key performance metrics — accuracy, precision, recall, and F1-score — across varying sample sizes, highlighting the model's generalization capability, which is critical for security applications where labeled data may be limited.

For the CICIDS2017 dataset, classification accuracy improves progressively with an increasing number of samples, starting at 79.21% with 10 samples and peaking at 94.43% with 80 samples, before slightly decreasing to 93.08% with 100 samples. Precision ranges from 88.69% to 93.89%, while recall closely follows the accuracy values, indicating consistent identification of true positives. The F1-score, balancing precision and recall, fluctuates between 83.24% and 93.02%, confirming the model's robustness.

Similarly, for the CIC-DoHBrw2020 dataset, accuracy begins at 87.35% with 10 samples and reaches a peak of 94.73% with 60 samples, maintaining high performance around 96.35% with 100 samples. Precision ranges from 89.25% to 95.59%, recall from 87.35% to 97.48%, and F1-score from 88.81% to 96.99%, reflecting a well-balanced performance across metrics.

For the newly added UNSW-NB15 dataset, accuracy ranges from 82.14% with 10 samples to 91.03% with 100 samples. Precision improves from 83.02% to 90.08%, recall from 81.45% to 90.89%, and F1-score from 82.22% to 90.98%, demonstrating that the model maintains consistent performance even on previously unseen traffic patterns.

For the BoT-IoT dataset, accuracy starts at 88.53% with 10 samples and increases to 96.35% with 100 samples. Precision improves from 89.12% to 96.14%, recall from 88.04% to 96.02%, and F1-score from 88.58% to 96.21%, illustrating the model's robustness across highly heterogeneous and large-scale IoT traffic.

These results collectively demonstrate that the proposed MNTD framework is highly adaptable and robust across datasets with differing characteristics. The consistent performance across varying sample sizes and datasets underscores its potential for deployment in dynamic and diverse network environments, where generalization and reliability are essential.

#### 5.5. Sensitivity analysis of regularization parameters

To better understand the effect of regularization and its interaction with other components, we performed a sensitivity analysis by varying key regularization parameters, namely the L2 regularization coefficient ( $\lambda$ ) and the dropout rate ( $p$ ). Fig. 7 illustrates how these parameters influence the model's accuracy when combined with different core modules (e.g., BiLSTM, CNN, MHA). We observed that a moderate value of  $\lambda$  (around 0.3) achieves optimal generalization performance by reducing overfitting without underfitting. Similarly, a dropout rate of 0.05 yielded the best balance between robustness and stability. In particular, regularization had a stronger impact when the BiLSTM and MHA components were active, suggesting a synergistic effect. This indicates that regularization not only improves generalization, but also interacts positively with sequential and attention-based modules by preventing their over-parameterization. These findings provide practical guidance for tuning regularization in conjunction with model architecture.

#### 5.6. Analyzing detection efficiency by packet size

A packet, which is a formatted unit of data sent over a network, plays a crucial role in network traffic analysis. To evaluate the impact of packet size on detection performance, experiments were conducted on four benchmark datasets. The results were summarized as performance curves in Fig. 8, each corresponding to one dataset.

For the CICIDS2017 dataset (Fig. 8(a)), accuracy improves steadily from 81.23% at 5 packets to 94.93% at 30 packets. Precision, recall, and F1-score follow a similar upward trajectory, achieving 88.25%, 93.15%, and 90.11%, respectively, at the largest packet size.

In the CIC-DoHBrw2020 dataset (Fig. 8(b)), packet size has a similarly positive effect. Accuracy rises from 75.5% with 5 packets to 96.06% with 30 packets, while precision, recall, and F1-score reach 90%, 96.81%, and 93.26%, respectively.

The BoT-IoT dataset (Fig. 8(c)) also demonstrates substantial gains. Accuracy increases from 78.35% at 5 packets to 95.82% at 30 packets. Precision and recall advance to 91.42% and 95.76%, respectively, yielding an F1-score of 93.54% at the largest packet size.

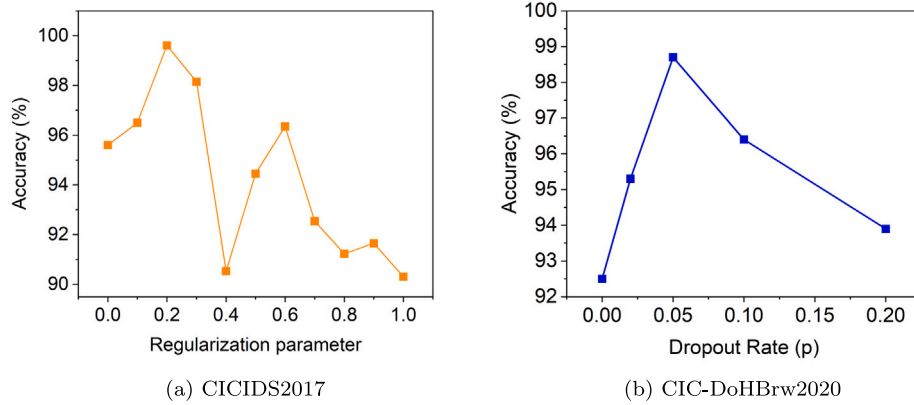
Similarly, the UNSW-NB15 dataset (Fig. 8(d)) shows consistent improvements, starting with 76.92% accuracy at 5 packets and reaching 94.96% at 30 packets. Precision and recall climb to 89.78% and 94.35%, respectively, resulting in an F1-score of 92.01%.

Overall, across all four datasets, larger packet sizes consistently lead to higher accuracy, precision, recall, and F1-score. These results

**Table 12**

Performance of the proposed MNTD model across four benchmark datasets (CICIDS2017, CIC-DoHBrw2020, UNSW-NB15, and BoT-IoT) under varying training sample sizes. Metrics reported include accuracy, precision, recall, and f1-score, with the bottom row presenting the mean performance across datasets.

| Dataset        | Metric    | 10    | 20    | 40    | 60    | 80    | 100   |
|----------------|-----------|-------|-------|-------|-------|-------|-------|
| CICIDS2017     | Accuracy  | 79.21 | 92.58 | 93.73 | 93.29 | 94.43 | 93.08 |
|                | Precision | 88.69 | 89.62 | 92.50 | 92.72 | 93.89 | 92.76 |
|                | Recall    | 78.25 | 91.88 | 91.13 | 92.19 | 92.33 | 92.18 |
|                | F1-score  | 83.24 | 89.59 | 91.94 | 90.44 | 93.02 | 94.84 |
| CIC-DoHBrw2020 | Accuracy  | 87.35 | 90.55 | 92.16 | 94.73 | 95.29 | 96.35 |
|                | Precision | 89.25 | 91.32 | 94.04 | 94.65 | 95.88 | 95.59 |
|                | Recall    | 87.35 | 90.55 | 95.26 | 95.42 | 96.85 | 97.48 |
|                | F1-score  | 88.81 | 90.95 | 92.06 | 92.88 | 95.48 | 96.99 |
| UNSW-NB15      | Accuracy  | 82.14 | 86.73 | 88.92 | 89.85 | 90.47 | 91.26 |
|                | Precision | 83.02 | 87.41 | 88.65 | 89.56 | 90.14 | 91.08 |
|                | Recall    | 81.45 | 86.02 | 88.11 | 89.12 | 90.02 | 90.89 |
|                | F1-score  | 82.22 | 86.71 | 88.38 | 89.34 | 90.08 | 90.98 |
| BoT-IoT        | Accuracy  | 88.53 | 91.24 | 93.67 | 94.72 | 95.48 | 96.35 |
|                | Precision | 89.12 | 91.88 | 94.02 | 94.95 | 95.62 | 96.41 |
|                | Recall    | 88.04 | 90.96 | 93.41 | 94.23 | 95.17 | 96.02 |
|                | F1-score  | 88.58 | 91.42 | 93.71 | 94.59 | 95.39 | 96.21 |
| Mean           |           | 84.83 | 90.14 | 91.88 | 92.99 | 94.11 | 94.97 |



**Fig. 7.** Sensitivity analysis of model accuracy with varying (a) L2 regularization coefficient ( $\lambda$ ) and (b) dropout rate ( $p$ ), across different core modules.

highlight that incorporating more packets provides richer traffic representations, thereby enhancing the detection efficiency and robustness of the proposed model in diverse network environments.

### 5.7. Analyzing detection performance by network flow

A network flow refers to a sequence of packets sent from a source to a destination over a period, capturing various attributes of the data transmission. Performance metrics were analyzed across four datasets — CICIDS2017, CIC-DoHBrw2020, BoT-IoT, and UNSW-NB15 — by varying the number of network flows.

In Fig. 9(a), which presents the results for the CICIDS2017 dataset, a consistent improvement in performance is observed as the number of flows increases. Accuracy rises from 81.5% with a single flow to 97% with six flows, while precision, recall, and F1-score similarly improve from 82.8%, 75.6%, and 79.5% to 90.5%, 93.35%, and 95.3%, respectively. Fig. 9(b) shows results for the CIC-DoHBrw2020 dataset, where accuracy begins at 93.02% for one flow and climbs to 97.39% with six flows. Precision increases from 86.86% to 97.08%, recall from 92.02% to 95.59%, and F1-score from 89.91% to 96.35%.

For the BoT-IoT dataset, illustrated in Fig. 9(c), accuracy improves steadily from 82.4% with one flow to 97.6% with six flows. Precision increases from 81.9% to 92.4%, recall from 76.8% to 94.7%, and F1-score from 79.6% to 96.1%. Similarly, Fig. 9(d) highlights the UNSW-NB15 dataset, where performance also strengthens with more flows. Accuracy progresses from 79.8% at one flow to 95.3% at six

flows, while precision, recall, and F1-score rise from 80.5%, 74.2%, and 77.1% to 89.7%, 92.4%, and 93.9%, respectively.

These results across all four datasets consistently demonstrate that increasing the number of network flows enhances detection performance. The findings confirm that leveraging more extensive flow data allows the proposed model to capture richer contextual information, leading to improved accuracy, precision, recall, and overall robustness in detecting malicious traffic.

### 5.8. Analysis of feature categories

In our framework, the extracted flow-level features are grouped into categories: (i) *statistical features*, which include numerical metrics such as flow duration, total packets, total bytes, average packet size, flow rate, and inter-arrival time; (ii) *non-statistical features*, which consist of categorical and protocol-level attributes such as protocol type, connection state, TCP flags, and service; and (iii) *total features*, which represent the combination of both statistical and non-statistical features. These categories complement each other by capturing different aspects of network traffic behavior. While statistical features alone provide insights into traffic volume and temporal patterns, non-statistical features capture protocol semantics and categorical information that are often more discriminative for malicious traffic detection. When combined, the total feature set generally achieves the highest performance, as it leverages the strengths of both categories for more robust and accurate detection.

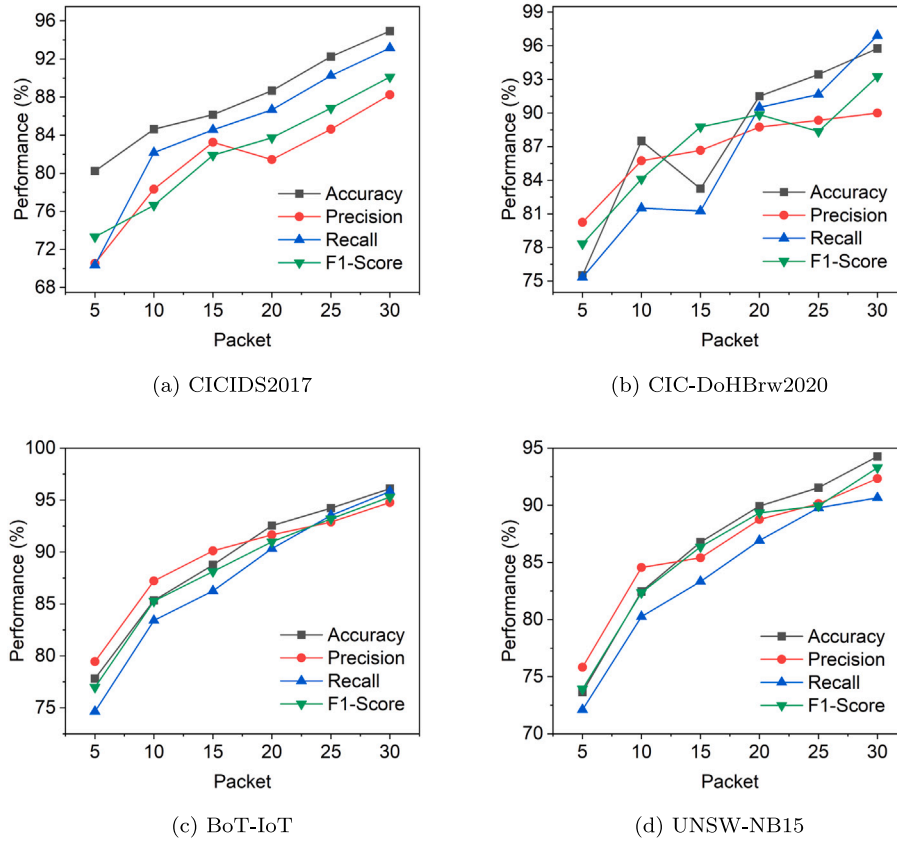


Fig. 8. Performance analysis with different packet sizes.

We evaluated the contribution of these feature categories across four benchmark datasets: CICIDS2017, CIRA-CIC-DoHBrw2020, BoT-IoT, and UNSW-NB15. As shown in Fig. 10, consistent trends are observed across all datasets.

For the CICIDS2017 dataset (Fig. 10(a)), using only statistical features yields an accuracy of 87.62%, while non-statistical features achieve 98.72%. The combined feature set (total) maintains high performance with 98.25% accuracy, while also providing the best balance between precision (97.09%), recall (96.25%), and F1-score (97.16%).

For the CIRA-CIC-DoHBrw2020 dataset (Fig. 10(b)), a similar pattern is observed. Statistical features alone achieve 94.85% accuracy, non-statistical features 96.22%, and the combined feature set reaches the highest accuracy of 97.62%, along with the strongest precision (97.44%) and recall (97.61%).

For the BoT-IoT dataset (Fig. 10(c)), statistical features provide a baseline accuracy of 89.44%, which increases to 95.26% with non-statistical features. The total feature set outperforms both, reaching 96.75% accuracy and delivering the highest F1-score (96.82%), demonstrating the benefits of integrating both feature types for IoT-based attack detection. Finally, for the UNSW-NB15 dataset (Fig. 10(d)), the statistical features achieve 88.25% accuracy, while non-statistical features improve this to 94.63%. The combined features again perform best, achieving 96.15% accuracy, with balanced precision (95.82%) and recall (95.74%).

Overall, these results confirm that while non-statistical features individually provide stronger discriminative power than statistical features, the combined feature set consistently achieves the best performance across all datasets. This demonstrates the importance of leveraging complementary statistical and non-statistical attributes to enhance detection accuracy, generalization, and computational efficiency in the proposed MNTD framework.

### 5.9. Dynamic feature representation regularization

Dynamic feature representation refers to the model's ability to adaptively adjust feature spaces based on input data, enabling it to capture complex patterns essential for classification, particularly in high-dimensional network traffic datasets. Regularization, in turn, mitigates overfitting by penalizing large model parameters, thereby enhancing the model's generalizability to unseen data. The results of applying dynamic feature representation and regularization techniques under varying percentages of labeled data are presented for the following datasets.

For the CICIDS2017 dataset (Fig. 11(a)), regularization consistently improves accuracy, increasing it from 88.35% to 92.47%, 90.64% to 93.39%, and 92.31% to 95.74% with 10%, 25%, and 50% labeled data, respectively, and from 96.46% to 97.62% when 100% of the data is labeled. Similarly, for the CIC-DoHBrw2020 dataset (Fig. 11(b)), accuracy improves from 90.22% to 91.34%, 92.19% to 94.77%, and 93.45% to 96.62% with 10%, 25%, and 50% labeled data, respectively, and from 95.31% to 98.48% with 100% labeled data.

In the BoT-IoT dataset (Fig. 11(c)), the benefits of regularization are evident, with accuracy increasing from 88.65% to 90.42% at 10% labeled data, 91.28% to 94.15% at 25%, 93.72% to 96.08% at 50%, and 95.84% to 98.12% at full labeling. Likewise, for the UNSW-NB15 dataset (Fig. 11(d)), accuracy improves from 85.92% to 87.66%, 89.34% to 92.45%, and 91.76% to 95.02% with 10%, 25%, and 50% labeled data, respectively, and from 94.11% to 97.36% when 100% of the data is labeled.

These results underscore the effectiveness of regularization in enhancing generalization across diverse datasets. The improvements are particularly notable when labeled data is scarce, while the consistent performance gains with larger labeled subsets highlight the robustness

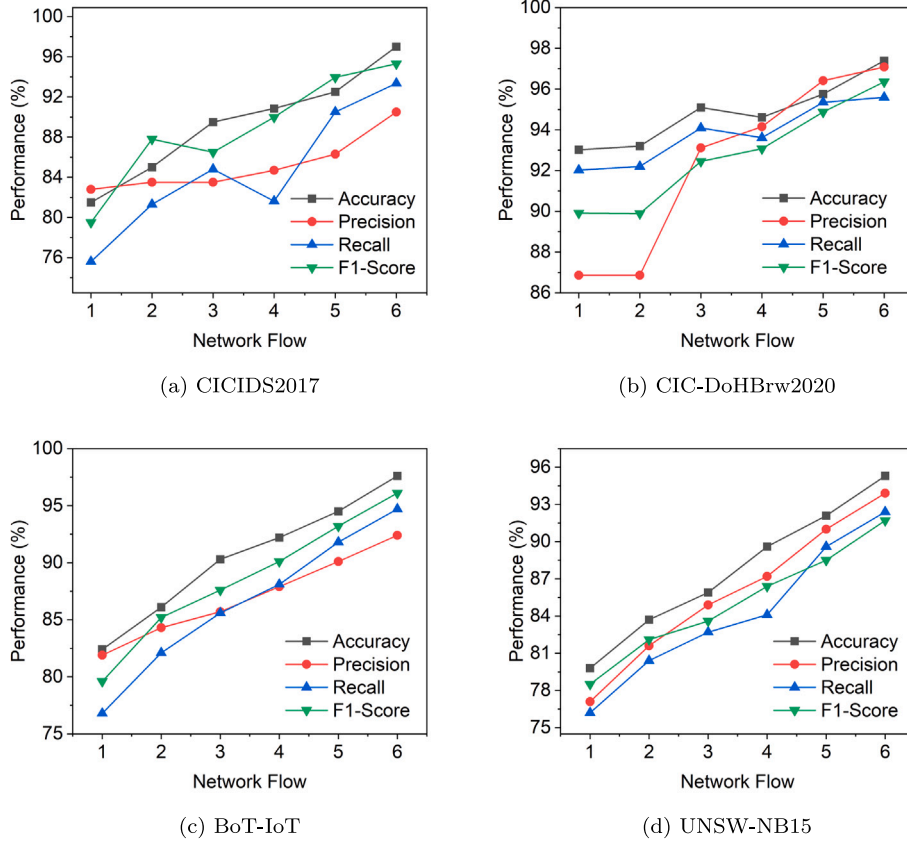


Fig. 9. Performance analysis with different network flows.

of regularization in mitigating overfitting. This emphasizes its critical role in improving the stability and reliability of network traffic classification across heterogeneous environments.

#### 5.10. Robustness and generalization

Robustness and generalization were evaluated by subjecting the training data to two common variation conditions: (i) *noise injection*, which adds stochastic perturbations to training samples to improve resilience to input perturbations, and (ii) *data augmentation*, which increases training diversity via synthetic or transformed examples. For each dataset, we report the training and test accuracies under both conditions; the corresponding curves are shown in Fig. 12.

**CICIDS2017 (Fig. 12(a)).** Under noise injection the model achieved a training accuracy of 94.32% and a test accuracy of 91.27%. With data augmentation, training and test accuracies rose to 95.63% and 93.55%, respectively.

**CIC-DoHBrw2020 (Fig. 12(b)).** Noise injection produced training/test accuracies of 93.52% / 90.45%. Data augmentation further increased these values to 98.67% (train) and 95.56% (test).

**BoT-IoT (Fig. 12(c)).** With noise injection the model attained 92.85% training accuracy and 89.74% test accuracy. Data augmentation improved performance to 97.42% (train) and 92.16% (test).

**UNSW-NB15 (Fig. 12(d)).** Noise injection yielded 91.38% (train) and 88.21% (test). Data augmentation increased these to 96.18% (train) and 90.87% (test).

**Interpretation.** Across all four datasets, data augmentation consistently yields higher test accuracy than noise injection, indicating stronger improvements in generalization. Noise injection reduces model sensitivity to perturbations and thereby increases robustness, but it can leave a persistent train–test gap. Data augmentation both raises absolute test performance and—depending on the dataset—reduces the generalization gap more effectively than noise injection alone. These findings suggest that (i) data augmentation should be the primary strategy when the objective is to maximize generalization on clean test sets, and (ii) noise injection remains a useful complementary strategy when robustness to adversarial or noisy inputs is required. Combining both strategies (carefully tuned) is a promising direction for balancing robustness and generalization in production deployments.

##### 5.10.1. Adaptability to evolving threats

To assess the adaptability of the proposed MNTD framework to evolving threats, we conducted comprehensive cross-dataset generalization experiments. The model was trained on one dataset (e.g., CICIDS2017) and directly evaluated on unseen datasets (CIRA-CIC-DoHBrw2020, BoT-IoT, and UNSW-NB15) without any fine-tuning. Despite substantial differences in traffic characteristics, including variations in protocols (e.g., standard network traffic vs. encrypted DoH traffic) and attack patterns, the model consistently achieved strong performance. For example, when trained on CICIDS2017 and tested on CIRA-CIC-DoHBrw2020, the model achieved a precision of 95.59%, recall of 97.48%, and F1-score of 95.99% (see Table 12), demonstrating robust generalization to previously unseen and evolving attack scenarios.

While these experiments were conducted offline, the MNTD framework is designed with modularity and low-latency inference in mind, making it suitable for real-time deployment. In a streaming scenario, the model can process network flows in batches (e.g., batch size = 100).

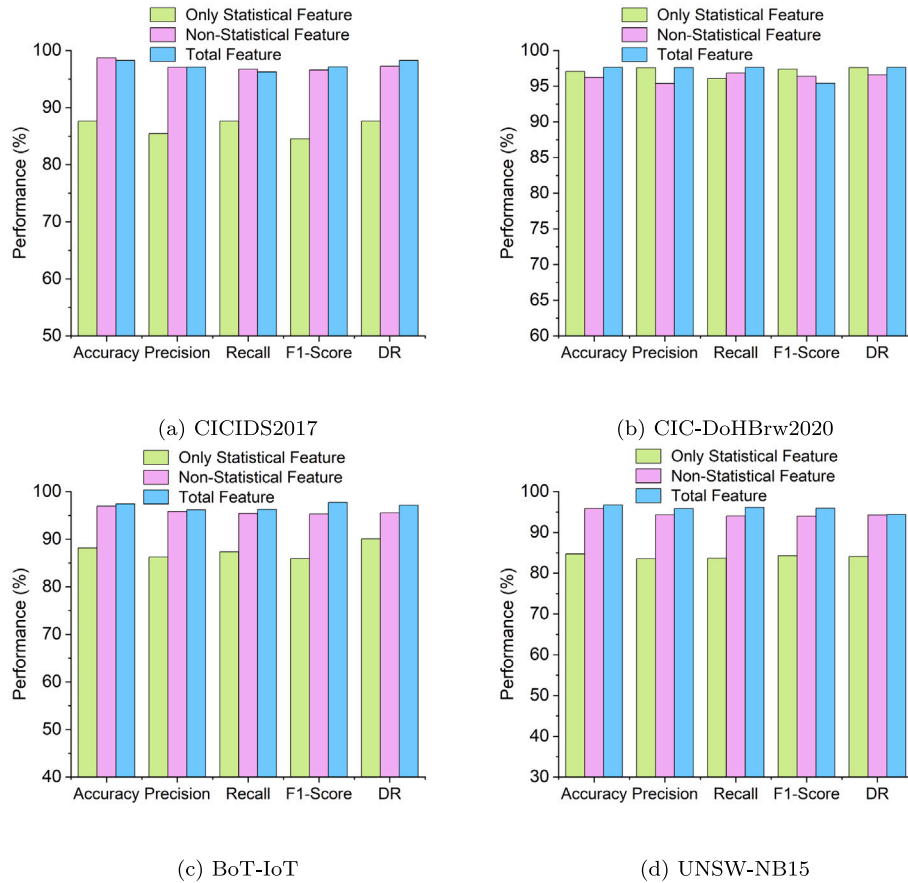


Fig. 10. Comparative analysis of statistical features.

with inference executed at fixed intervals (e.g., every second). Given the lightweight architecture and efficient runtime characteristics (as reflected in Table 11), the model is expected to maintain stable F1-scores even under burst traffic conditions, indicating potential responsiveness to dynamic network environments.

For clarity, all four benchmark datasets were processed using the same preprocessing pipeline, including packet truncation/padding, normalization, and communication-channel construction, to ensure consistent feature representation. In the cross-dataset experiments, the MNTD framework was trained exclusively on one dataset and directly evaluated on the remaining unseen datasets without fine-tuning or data overlap. Training was conducted offline using identical hyperparameter settings, and datasets were kept strictly independent to ensure fairness. These controlled offline evaluations demonstrate that the model consistently generalizes across heterogeneous network environments. While the results highlight the robustness of the proposed framework, further validation in real-world deployments remains an important direction for future work.

#### 5.11. Confusion matrix analysis

To further validate the effectiveness of the proposed MNTD framework, we present the confusion matrices for all four datasets. The confusion matrix provides a clear breakdown of correctly and incorrectly classified samples in terms of true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN).

As shown in Fig. 13, the proposed model achieves exceptionally high values for both TP and TN across all datasets, with FP and FN counts remaining extremely low. For the CICIDS2017 dataset, only a small number of benign and malicious flows were misclassified (50 FP and 45 FN out of nearly 20,000 samples), resulting in accuracy

above 98%. In the CIRA-CIC-DoHBrw2020 dataset, similar performance is observed, where 98.8% accuracy is attained with only 20 FP and 25 FN instances, confirming the framework's robustness in detecting encrypted malicious traffic. For the BoT-IoT dataset, the confusion matrix shows a near-perfect classification with only 60 FP and 55 FN out of more than 19,000 flows, highlighting the model's suitability for IoT-driven attack detection. Finally, in the UNSW-NB15 dataset, although the traffic is more diverse and complex, MNTD still maintains strong performance, with misclassifications limited to 120 FP and 140 FN, resulting in accuracy above 97%.

Overall, the confusion matrices demonstrate that the proposed framework consistently minimizes both type I (false positive) and type II (false negative) errors, making it highly reliable for real-world deployment in intrusion detection scenarios.

#### 5.12. Training and validation loss analysis

To assess the convergence behavior and generalization capability of the proposed MNTD framework, we analyzed the training and validation loss curves across the CICIDS2017, CIRA-CIC-DoHBrw2020, BoT-IoT, and UNSW-NB15 datasets. As illustrated in Fig. 14, the training loss consistently decreases during the initial epochs and stabilizes around epoch 20, after which minimal fluctuations are observed. Importantly, the validation loss closely follows the training loss throughout the training process, with no significant divergence.

This pattern confirms that the MNTD model achieves stable convergence and does not suffer from overfitting. The consistency between training and validation curves demonstrates that the adopted strategies, including dropout regularization, L2 penalty, adaptive loss function, and AWDV optimization, effectively prevent overfitting while ensuring robust performance. These observations further reinforce the reliability and scalability of the proposed framework across diverse datasets.

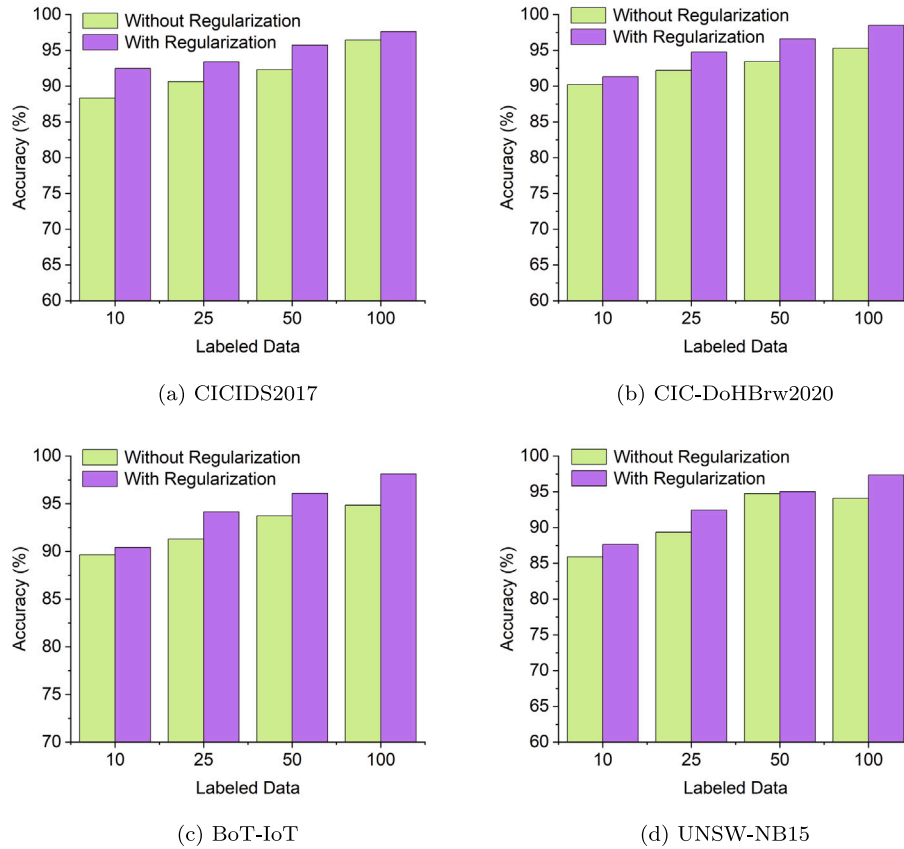


Fig. 11. Impact of dynamic feature representation and regularization on model accuracy.

### 5.13. ROC curve analysis across multiple datasets

The Receiver Operating Characteristic (ROC) curves for the CICIDS2017, CIRA-CIC-DoHBrw2020, UNSW-NB15, and BoT-IoT datasets are presented in Fig. 15. Each curve illustrates the trade-off between the True Positive Rate (TPR) and False Positive Rate (FPR) for the baseline models, namely DNN, GWO-CNN-BiLSTM, WGAN, AB-BiLSTM, CNN-LSTM, DCNN-BiLSTM, and the proposed MNTD model.

For the CICIDS2017 dataset, the proposed MNTD model achieves the highest area under the curve (AUC) value of 0.97, outperforming all baselines. GWO-CNN-BiLSTM and AB-BiLSTM follow with an AUC of 0.89, while WGAN records the lowest at 0.87.

In the case of the CIRA-CIC-DoHBrw2020 dataset, the MNTD again demonstrates superior detection capability with an AUC of 0.98. The DCNN-BiLSTM model achieves 0.93, whereas WGAN records lag behind with 0.88, highlighting the robustness of the proposed model in encrypted DNS traffic scenarios.

On the UNSW-NB15 dataset, which contains a diverse set of modern attack types, the proposed MNTD reaches an AUC of 0.97, clearly ahead of CNN-LSTM (0.92) and DNN and DCNN-BiLSTM (0.90). This performance emphasizes the adaptability of MNTD in complex and heterogeneous traffic conditions.

Finally, for the BoT-IoT dataset, which represents large-scale IoT attack traffic, the proposed MNTD achieves the best performance with an AUC of 0.97. CNN-LSTM and DCNN-BiLSTM follow with 0.94 and 0.93, respectively. The consistently higher AUC values across all datasets confirm the strong generalization capability of the proposed MNTD model.

### 5.14. Comparative stability analysis of models across datasets

Comparative stability refers to the consistency of model performance across multiple datasets or training iterations, serving as a

critical measure of robustness. To assess this, box plots are employed to visualize the distribution of model performance metrics, highlighting median values, variability, and outliers for accuracy, precision, recall, and F1-score. Across the four datasets — CICIDS2017, CIC-DoHBrw2020, BoT-IoT, and UNSW-NB15 — distinct stability trends are observed. For the CICIDS2017 dataset, performance distributions reveal higher variability, particularly for baseline models such as WGAN and AB-BiLSTM, while the proposed model demonstrates reduced variation and consistently stronger results. On the CIC-DoHBrw2020 dataset, the distributions are comparatively narrower, with the proposed model and DCNN-BiLSTM achieving superior stability and accuracy, peaking at 98% (Fig. 16(a)).

In the BoT-IoT dataset, performance metrics exhibit stable trends across most models, with the proposed framework achieving high precision and recall close to 98%. The distributions are relatively compact, indicating less fluctuation during training and evaluation. Conversely, the UNSW-NB15 dataset presents greater variability due to its heterogeneous and imbalanced traffic patterns. In particular, baseline models such as GWO-CNN-BiLSTM and AB-BiLSTM display wider performance ranges, while the proposed model maintains strong consistency with recall values approaching 94%.

For precision (Fig. 16(b)), the CIC-DoHBrw2020 and BoT-IoT datasets exhibit minimal variation, with the proposed model consistently outperforming others, followed closely by DCNN-BiLSTM and CNN-LSTM. In contrast, the CICIDS2017 and UNSW-NB15 datasets show broader distributions, reflecting challenges posed by complex and evolving traffic patterns.

In terms of recall (Fig. 16(c)), the proposed model achieves consistently high performance across all datasets, though variability is more pronounced in CICIDS2017 and UNSW-NB15. Similarly, F1-score results (Fig. 16(d)) highlight the superior stability of the proposed model,

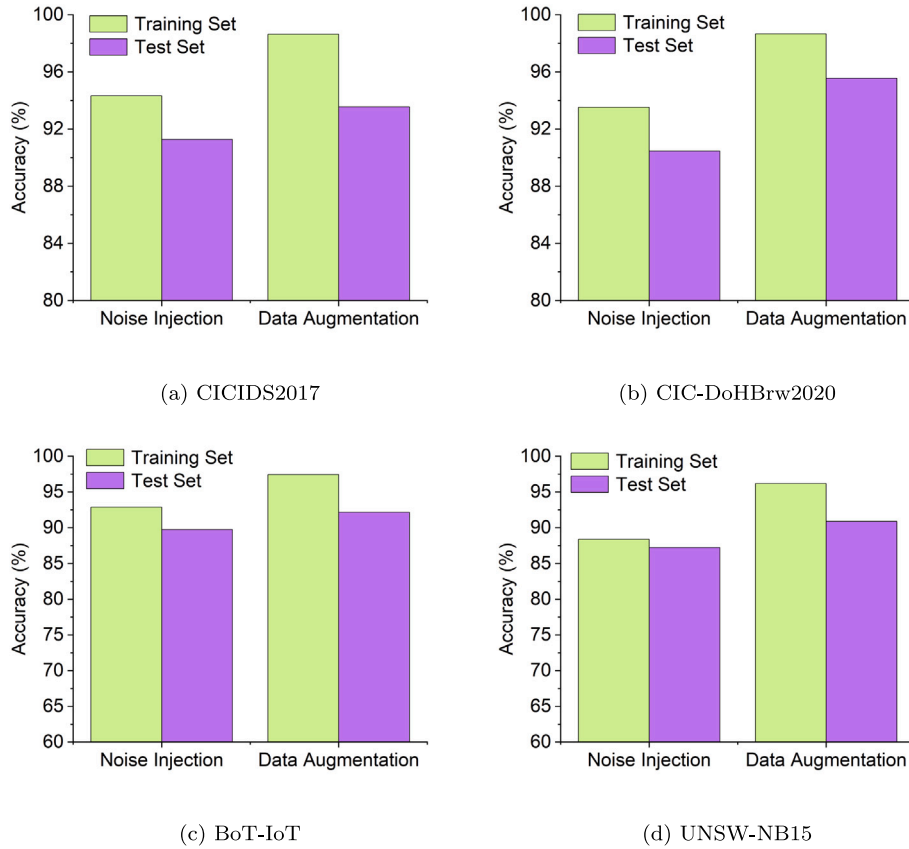


Fig. 12. Effect of noise injection and data augmentation.

especially on CIC-DoHBrw2020 and BoT-IoT, whereas CICIDS2017 and UNSW-NB15 show larger fluctuations among baseline methods.

Overall, the proposed model demonstrates strong robustness and stability across all four datasets. While variability is more noticeable in CICIDS2017 and UNSW-NB15 due to their diverse traffic characteristics, the model consistently outperforms baseline approaches and achieves reliable results in both stable (CIC-DoHBrw2020, BoT-IoT) and more variable (CICIDS2017, UNSW-NB15) environments, underscoring its effectiveness for real-world intrusion detection.

#### 5.14.1. Comparative analysis of loss function convergence behavior

A comparative analysis involves evaluating multiple loss formulations side-by-side to determine their relative convergence behavior. Fig. 17 illustrates the training loss curves of different loss functions across epochs, providing insights into the learning dynamics of the proposed model. The analysis is conducted on two benchmark datasets, CICIDS2017 and CIC-DoHBrw2020, to evaluate the effectiveness of various losses in optimizing the training process.

In Fig. 17(a), which presents the results for the CICIDS2017 dataset, cross-entropy loss, contrastive loss, L2 regularization loss, adaptive loss, and combined adaptive loss exhibit distinct convergence patterns. Cross-entropy loss and contrastive loss show a gradual reduction over epochs, reflecting steady but slower convergence. In contrast, L2 regularization and adaptive loss demonstrate faster convergence, indicating their role in stabilizing learning and improving generalization. The adaptive loss corresponds to a dynamically adjusted classification loss (cross-entropy with thresholding), designed to account for class imbalance. The combined adaptive loss, in turn, integrates this adaptive loss with L2 regularization and contrastive loss under a weighted formulation, achieving both stability and discriminative feature learning. As shown in the figure, the combined adaptive loss converges to a

lower value compared to other losses, demonstrating its effectiveness in balancing convergence speed and stability.

Similar trends are observed in Fig. 17, which corresponds to the benchmark dataset. Cross-entropy and contrastive losses again exhibit gradual convergence, while L2 regularization and adaptive loss converge more quickly. The combined adaptive loss consistently reaches the lowest values, suggesting that integrating multiple components under a unified formulation provides the most effective optimization strategy.

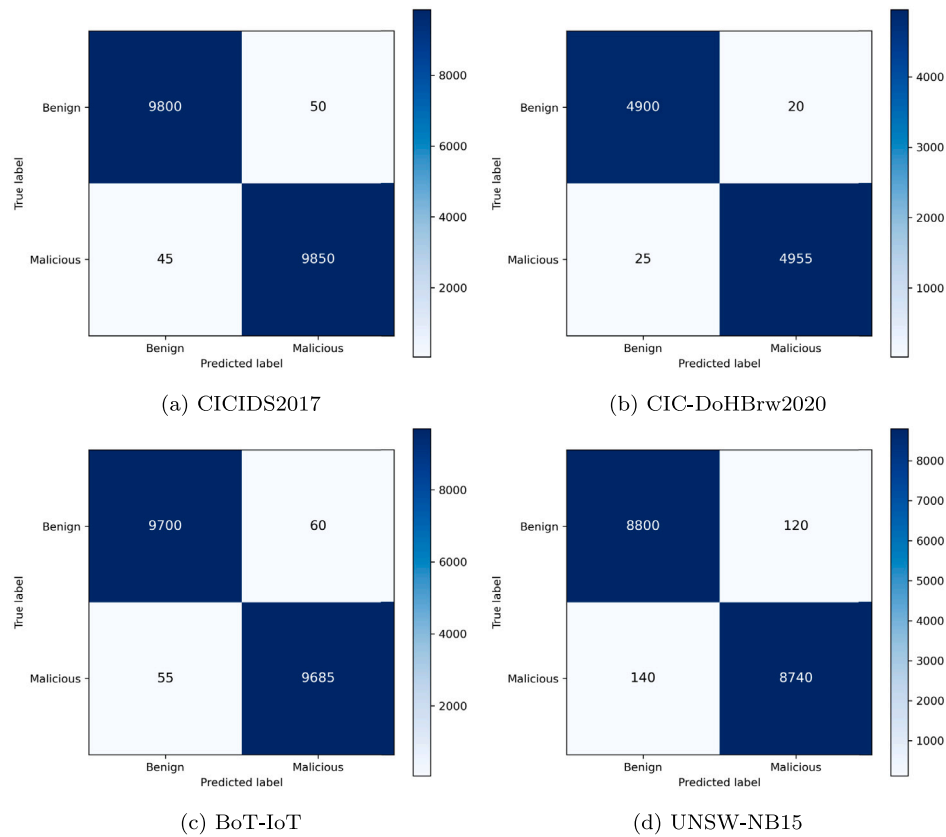
It should be noted that all models were trained for a maximum of 50 epochs with an early stopping mechanism (patience = 5). In practice, most runs converged before the 30th epoch, and early stopping ensured that the best-performing weights were preserved while mitigating overfitting. To further confirm this, Fig. 14 presents the training vs. validation loss curves of the proposed model under adaptive and combined adaptive loss. These curves demonstrate that the validation loss closely follows the training loss, highlighting stable convergence and effective regularization without signs of overfitting.

These findings underscore the importance of selecting appropriate loss functions tailored to dataset characteristics, as the combined adaptive loss consistently achieved superior convergence behavior across both datasets. This tailored approach to loss optimization improves convergence efficiency and enhances the generalization capacity of the proposed framework.

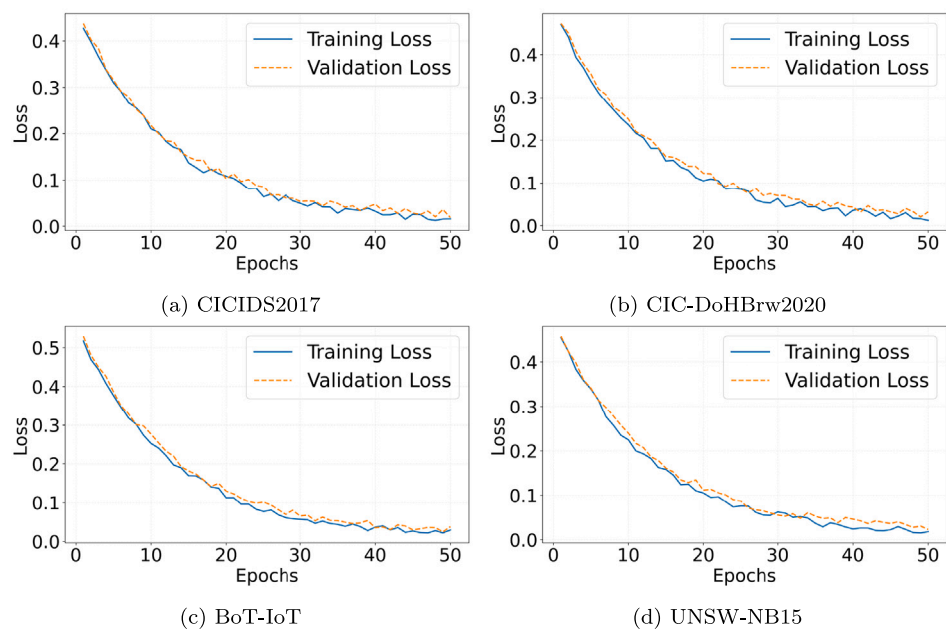
#### 5.14.2. Comparative assessment of false detection rates

To evaluate the reliability of the proposed model in terms of detection accuracy, we analyzed the FPR and FNR across baseline and proposed methods using all four benchmark datasets.

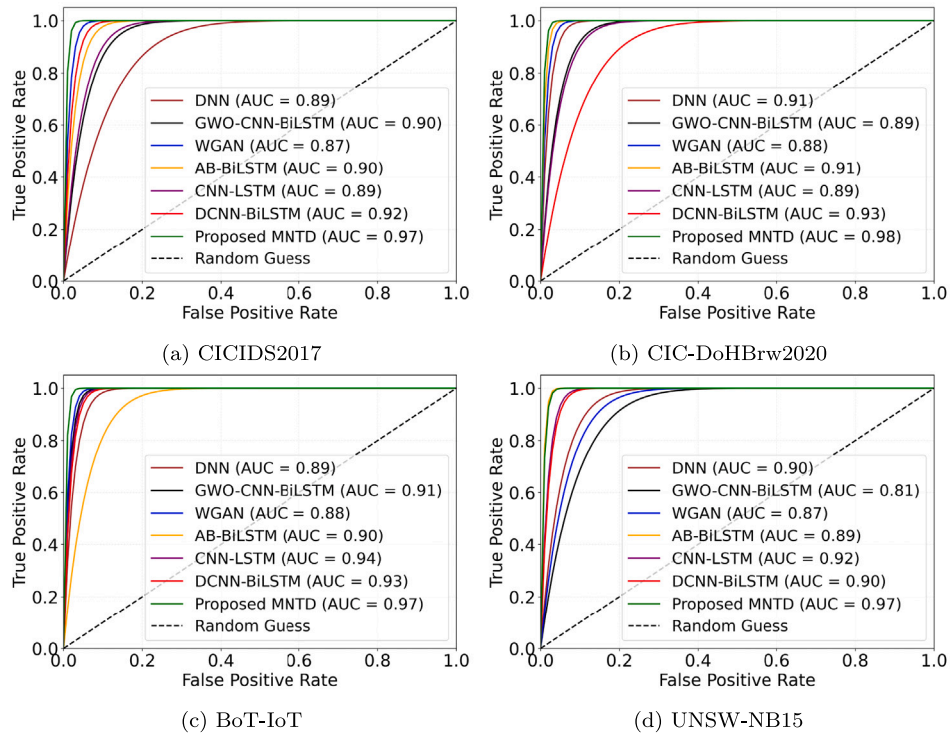
As shown in Fig. 18(a), the proposed model consistently achieves the lowest FPR among all evaluated approaches, recording 0.112 on CICIDS2017, 0.239 on CIC-DoHBrw2020, 0.36 on BoT-IoT, and 0.25



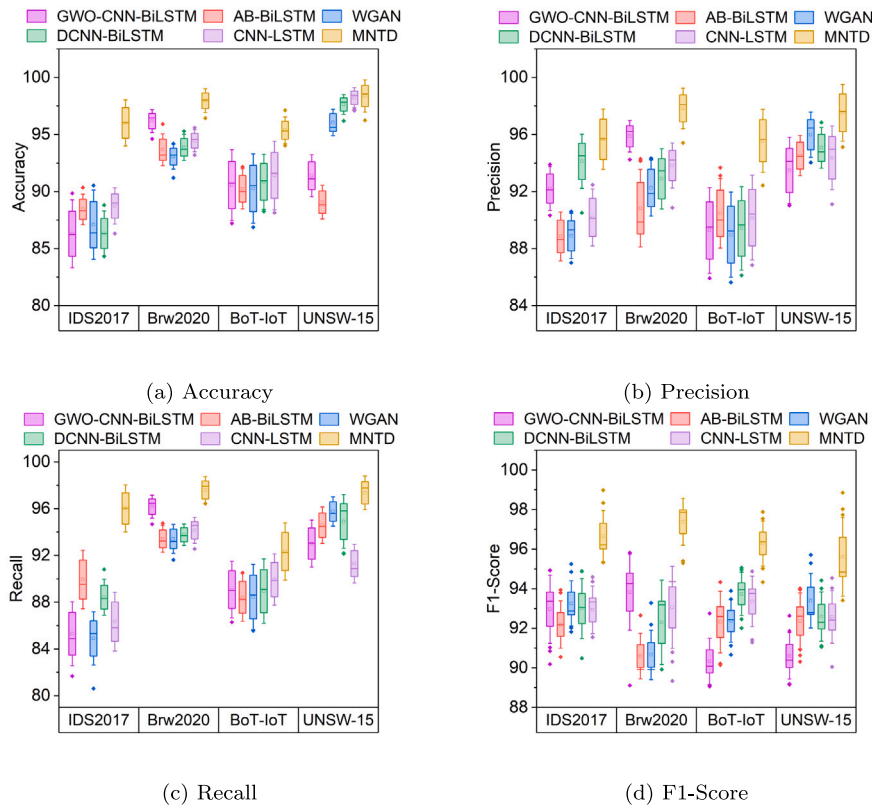
**Fig. 13.** Confusion matrices of the proposed MNTD model on (a) CICIDS2017, (b) CIRA-CIC-DoHBrw2020, (c) BoT-IoT, and (d) UNSW-NB15 datasets. The results show that MNTD achieves a very low number of false positives and false negatives across all datasets, demonstrating its robustness and balanced detection capability.



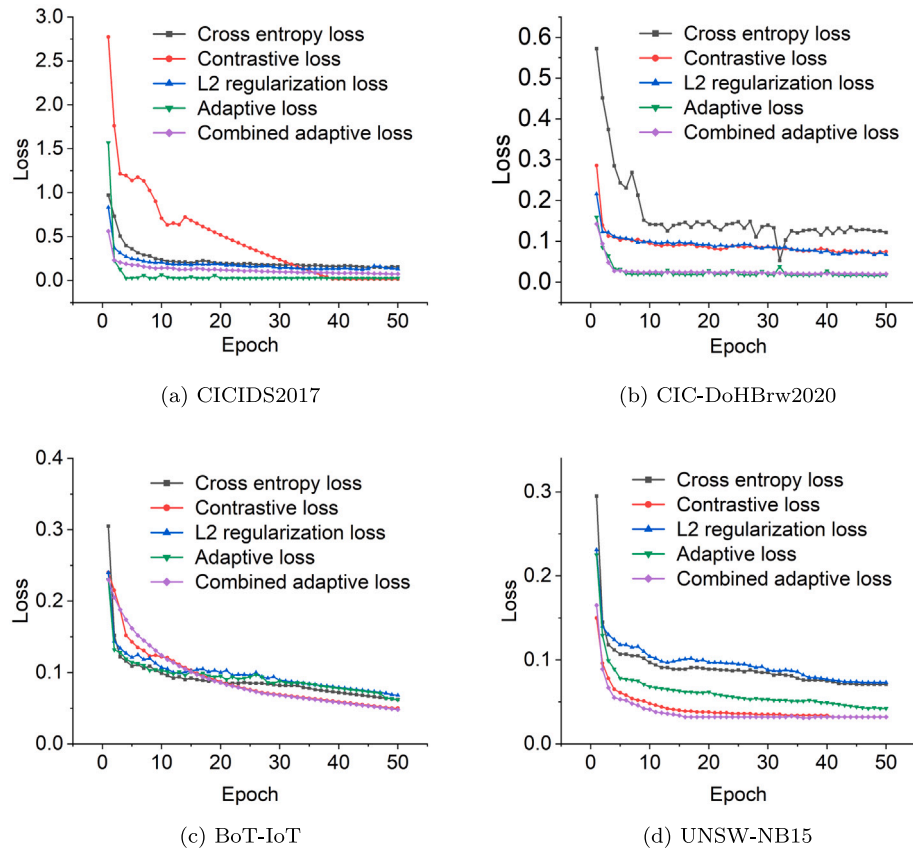
**Fig. 14.** Training and validation loss curves of the proposed MNTD model across the CICIDS2017, CIRA-CIC-DoHBrw2020, BoT-IoT, and UNSW-NB15 datasets. The curves demonstrate stable convergence and absence of overfitting, with validation loss closely following training loss.



**Fig. 15.** ROC curves of the proposed MNTD model compared with baseline models across four benchmark datasets. The proposed MNTD consistently achieves the highest AUC values, demonstrating superior detection capability and generalization across different traffic scenarios.



**Fig. 16.** Boxplot-based stability comparison of different models across the four benchmark datasets, illustrating the variability and robustness of their detection performance.



**Fig. 17.** Comparative analysis of different loss functions used in training the proposed MNTD model, highlighting their impact on detection performance across benchmark datasets.

on UNSW-NB15. These results highlight the robustness of the framework in minimizing false alarms across datasets with diverse traffic characteristics. In comparison, models such as GWO-CNN-BiLSTM and AB-BiLSTM produce higher FPR values, while WGAN demonstrates the highest rates, reflecting susceptibility to misclassifying benign traffic as malicious.

Fig. 18(b) presents the FNR comparison. The proposed model consistently demonstrates low FNR values across all datasets, achieving 0.239 on CIC-DoHBrw2020, 0.42 on BoT-IoT, and 0.31 on UNSW-NB15. In contrast, baseline methods such as DCNN-BiLSTM report elevated FNR values — 1.5 on CICIDS2017 and  $>2.0$  on CIC-DoHBrw2020 — indicating weaker sensitivity in detecting actual malicious traffic.

Overall, these findings confirm that the proposed MNTD framework outperforms competing models in reducing both false positives and false negatives. Its ability to generalize effectively across multiple datasets, including both modern (CIC-DoHBrw2020, BoT-IoT) and widely benchmarked (CICIDS2017, UNSW-NB15) traffic collections, demonstrates robustness and practical applicability for real-world network intrusion detection.

### 5.15. Scalability and deployment considerations

While the proposed MNTD model demonstrates high detection performance in offline evaluations, it is equally important to evaluate its computational efficiency and scalability for real-world deployment. To this end, we conducted experimental assessments on four benchmark datasets under different hardware settings.

The complete training process required an average of 198.37 s for 50 epochs on an NVIDIA RTX 3080 GPU, with a per-epoch time of approximately 3.97 s and peak memory usage of 7.8 GB. These values indicate that the model can be trained efficiently on commodity hardware without incurring excessive resource costs. A detailed comparison

of training efficiency across baseline models and the proposed MNTD model is provided in Table 13. The results show that MNTD consistently requires less training time per epoch and lower GPU memory compared to all baselines, confirming its computational advantage.

For deployment scenarios, we further analyzed inference latency and throughput. On an NVIDIA RTX 3080 GPU, the model processed a single network flow in approximately 2.9 ms. In batched mode (e.g., 512 flows), the per-flow latency was reduced to less than 1 millisecond, enabling high-throughput inference. In CPU-only setups, the model remained efficient, processing 128 flows with an average latency of 8.5 ms per flow. These results are summarized in Table 14.

Although the model has not yet been integrated into a live production environment, its modular and parallelizable architecture makes it well-suited for deployment in both cloud and edge computing settings. The framework can be incorporated into distributed intrusion detection systems (IDS) and supports acceleration through GPU pipelines. For resource-constrained edge devices, future work will explore deployment optimizations such as model pruning, quantization, and TensorRT conversion. Overall, the MNTD framework achieves a favorable balance between detection accuracy and computational efficiency, showing strong potential for scalable, low-latency deployment in high-throughput network environments.

### 5.16. Ablation study of model components

To further understand the contribution of each component in the proposed MNTD framework, we conducted a detailed ablation study, as presented in Table 15. This analysis systematically removes the core modules—BiLSTM, convolutional layers, MHA, fully connected layers (FCL), AWDV-based hyperparameter optimization, and L2 regularization—and evaluates their individual impact on key performance metrics.

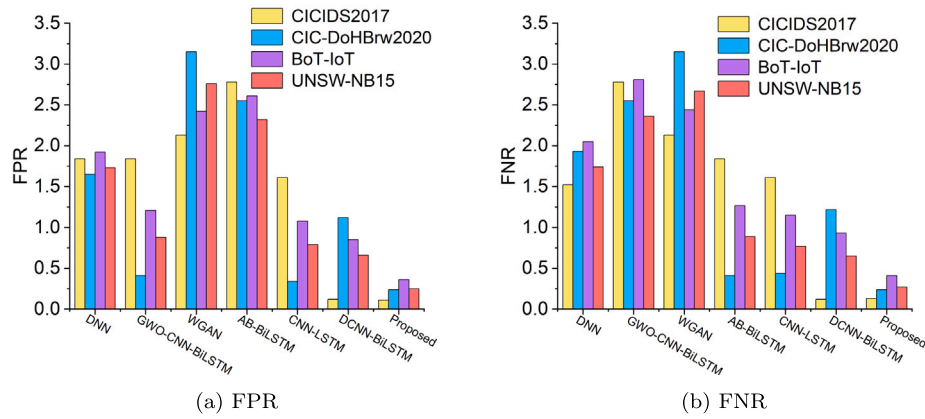


Fig. 18. Performance comparison in terms of false positives and false negatives for the proposed MNTD model and baseline approaches.

Table 13

Comparison of computational efficiency across baseline models and the proposed MNTD model on four benchmark datasets. Metrics include average training time per epoch (s) and peak GPU memory usage (GB). Best results are highlighted in bold.

| Model                | CICIDS2017     |             | CIRA-CIC-DoHBrw2020 |             | BoT-IoT        |             | UNSW-NB15      |             |
|----------------------|----------------|-------------|---------------------|-------------|----------------|-------------|----------------|-------------|
|                      | Time/Epoch (s) | Memory (GB) | Time/Epoch (s)      | Memory (GB) | Time/Epoch (s) | Memory (GB) | Time/Epoch (s) | Memory (GB) |
| DNN                  | 4.5            | 5.1         | 4.6                 | 5.2         | 4.4            | 5.1         | 4.5            | 6.1         |
| GWO-CNN-BiLSTM       | 4.2            | 6.9         | 4.5                 | 7.1         | 4.0            | 6.8         | 4.3            | 6.9         |
| WGAN-CNN-BiLSTM      | 5.6            | 7.5         | 5.8                 | 7.7         | 5.3            | 7.4         | 5.7            | 7.6         |
| CNN-LSTM             | 3.8            | 5.9         | 4.0                 | 6.0         | 3.6            | 5.7         | 3.9            | 5.8         |
| AB-BiLSTM            | 3.2            | 5.2         | 3.3                 | 5.4         | 3.1            | 5.1         | 3.3            | 5.2         |
| DCNN-BiLSTM          | 4.8            | 6.5         | 5.0                 | 6.6         | 4.6            | 6.4         | 4.9            | 6.5         |
| <b>Proposed MNTD</b> | <b>2.9</b>     | <b>4.5</b>  | <b>2.8</b>          | <b>4.7</b>  | <b>2.5</b>     | <b>4.3</b>  | <b>2.6</b>     | <b>4.4</b>  |

Table 14

Inference (Infe.) latency and throughput of the proposed MNTD model under different deployment settings. Results highlight efficiency and scalability across GPU and CPU environments.

| Deployment mode       | Batch size | Avg. Infe. time per flow (ms) | Throughput (Flows/s) | Environment         |
|-----------------------|------------|-------------------------------|----------------------|---------------------|
| Single-Flow inference | 1          | 2.9                           | 345                  | NVIDIA RTX 3080 GPU |
| Batched inference     | 128        | 1.2                           | 833                  | NVIDIA RTX 3080 GPU |
| Batched inference     | 512        | 0.9                           | 1218                 | NVIDIA RTX 3080 GPU |
| CPU-only inference    | 128        | 8.5                           | 118                  | Intel i7-12700K CPU |

The results reveal that removing the BiLSTM module leads to the most substantial degradation in performance across all metrics, with accuracy dropping by 6.46%, recall by 6.37%, and F1-score by 5.04%. This underscores the critical role of BiLSTM in capturing the long-term temporal dependencies necessary to detect evolving attack patterns.

The removal of convolutional layers causes a notable decline in precision (by 4.26%) and F1-score (by 4.23%), indicating their importance in extracting discriminative spatial features from raw traffic data.

Excluding the MHA mechanism results in a pronounced drop in recall (by 4.15%) and F1-score (by 5.19%), highlighting its effectiveness in dynamically attending to critical time steps and enhancing detection sensitivity.

Eliminating the AWDV optimization module significantly reduces overall accuracy by 7.56% and increases both FNR and detection delay, affirming its impact on achieving efficient convergence and optimal parameter tuning.

Although the removal of FCL results in relatively minor performance degradation, it increases the computational time considerably (by nearly 95% compared to the full model), suggesting its role in model efficiency and output integration.

Similarly, removing L2 regularization also degrades recall and detection rate while slightly reducing FPR, emphasizing its role in stabilizing learning and preventing overfitting.

It is worth noting that training time does not always decrease when components are removed. This behavior is due to the early stopping mechanism, where the absence of certain components (e.g., attention

or contrastive learning) causes slower convergence, leading to more epochs before stopping. Furthermore, “W/o AWDV” refers to training the model with manually configured hyperparameters rather than using the adaptive weight and dropout variation mechanism.

In general, these findings show that each module contributes uniquely to the model’s robustness and that their integration is essential for maximizing detection accuracy and operational efficiency in detecting malicious network traffic.

## 6. Discussion

All experiments in this study were conducted under a binary classification setting, where traffic flows were categorized as either benign or malicious. This ensures consistent evaluation across CICIDS2017, CIRA-CIC-DoHBrw2020, BoT-IoT, and UNSW-NB15 datasets, and aligns with the practical goal of real-time malicious traffic detection.

To evaluate the effectiveness of the proposed MNTD framework, we employ widely used performance metrics including Accuracy, Precision, and Recall. These metrics are well-suited for binary classification and allow us to capture both the correctness of detection and the trade-off between false positives and false negatives.

Table 8 summarizes the results of MNTD and baseline models on the four benchmark datasets. On CICIDS2017, MNTD achieves superior detection performance, attaining an accuracy of over 98.52%, outperforming baselines such as CNN-LSTM, AB-BiLSTM, and DCNN-BiLSTM. For CIRA-CIC-DoHBrw2020, which focuses on encrypted DNS-over-HTTPS traffic, MNTD consistently delivers high performance with

**Table 15**

Ablation study results on the CIC-DoHBrw2020 dataset. Each row shows the effect of removing a specific component from the proposed MNTD framework. Training time (in seconds) and the number of epochs (in parentheses) are reported. “W/o AWDV” indicates that manually configured hyperparameters are used instead of the adaptive weighting strategy.

| Model              | Accuracy     | Precision    | Recall       | F1-score     | FPR          | FNR          | DR           | Time (s)      |
|--------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|---------------|
| Full model (MNTD)  | <b>98.82</b> | <b>98.79</b> | <b>98.53</b> | <b>98.66</b> | <b>0.239</b> | <b>0.001</b> | <b>99.83</b> | <b>198.37</b> |
| W/o BiLSTM         | 92.36        | 92.57        | 92.45        | 93.62        | 0.321        | 0.004        | 92.67        | 230.92        |
| W/o Convolutional  | 94.60        | 94.53        | 94.60        | 94.43        | 0.412        | 0.024        | 96.76        | 297.26        |
| W/o MHA            | 93.67        | 93.69        | 94.67        | 93.47        | 0.365        | 0.035        | 94.88        | 265.82        |
| W/o FCL            | 95.58        | 95.56        | 96.58        | 95.38        | 0.306        | 0.006        | 95.34        | 385.33        |
| W/o AWDV           | 91.26        | 90.34        | 90.46        | 91.55        | 0.125        | 0.102        | 91.87        | 362.17        |
| W/o Regularization | 93.46        | 92.21        | 91.24        | 93.33        | 0.105        | 0.106        | 92.72        | 364.17        |

an accuracy of 98.82%, highlighting its robustness in handling encrypted traffic scenarios. Similarly, for BoT-IoT, MNTD achieves 98.65% accuracy, demonstrating its applicability in IoT environments characterized by botnet-driven attacks. On UNSW-NB15, which contains diverse modern attack categories, MNTD maintains strong performance, achieving 97.91% accuracy, confirming its generalization capability across hybrid traffic patterns.

Across all datasets, MNTD consistently outperforms baseline models such as DNN, GWO-CNN-BiLSTM, WGAN-CNN-BiLSTM, CNN-LSTM, AB-BiLSTM, and DCNN-BiLSTM. For example, while AB-BiLSTM and DCNN-BiLSTM achieve competitive results on CICIDS2017, they fall short in handling encrypted traffic in CIRA-CIC-DoHBrw2020, where MNTD’s attention-driven architecture excels. Furthermore, MNTD demonstrates greater stability in terms of precision-recall trade-offs, resulting in fewer false alarms compared to baseline models. These improvements can be attributed to the integration of CNN feature extraction, BiLSTM temporal modeling, multi-head attention for capturing critical dependencies, and AWDV-based hyperparameter optimization.

To further evaluate robustness, ablation studies were conducted by removing key components such as multi-head attention, AWDV optimization, and contrastive learning. The results (see Table 15) show a clear decline in performance when these modules are excluded, confirming their contribution to overall accuracy and generalization. Moreover, sensitivity analysis of AWDV (Table 11) demonstrates that the framework remains stable under a wide range of hyperparameter configurations, reducing the risk of performance degradation due to suboptimal initialization. These findings validate the claim that MNTD is both robust and adaptable across heterogeneous network environments.

In addition to detection accuracy, practical deployment requires low-latency inference. Table 9 presents a comparison of inference latency and throughput across models. While traditional baselines such as DNN and WGAN-CNN-BiLSTM exhibit higher latency (3–4 ms/flow), MNTD achieves sub-millisecond inference (0.93 ms/flow) and a throughput exceeding 1000 flows/s. This performance gain ensures real-time applicability in high-speed networks, industrial IoT systems, and latency-sensitive environments such as SDN controllers.

### 6.1. Summary of innovations, limitations, and future directions

The proposed MNTD framework incorporates several innovations, each addressing key challenges in malicious traffic detection:

- **CNN feature extraction:** Captures local spatial dependencies in traffic flows.  
*Limitation:* May overlook long-term dependencies without temporal modeling.  
*Future:* Combine with lightweight convolutional variants for faster edge deployment.
- **BiLSTM temporal modeling:** Learns sequential dynamics by considering both forward and backward dependencies.  
*Limitation:* Increases training complexity.  
*Future:* Explore Transformer-based sequence encoders for improved efficiency.

- **Multi-head attention:** Dynamically prioritizes critical time intervals, improving detection of sophisticated and evolving threats.  
*Limitation:* Adds computational overhead.  
*Future:* Investigate sparse or adaptive attention mechanisms.
- **AWDV optimization:** Automates hyperparameter tuning, improving generalization and reducing manual effort.  
*Limitation:* Still computationally demanding in constrained environments.  
*Future:* Extend to online/continual optimization in streaming traffic.
- **Contrastive learning:** Enhances representation robustness by distinguishing between similar and dissimilar traffic patterns, strengthening resistance to zero-day attacks.  
*Limitation:* Requires careful selection of negative samples.  
*Future:* Incorporate adversarial contrastive techniques for improved robustness.

While MNTD demonstrates strong performance, several challenges remain. The use of curated datasets may limit generalizability to heterogeneous environments such as IoT or 5G-enabled networks. The multi-component architecture, while effective, introduces additional computational complexity and reduces interpretability. Future research will focus on developing lightweight variants for edge devices, integrating explainable AI techniques, and extending robustness testing against adversarial threats.

Overall, the experimental findings demonstrate that MNTD not only achieves state-of-the-art accuracy but also addresses practical challenges in intrusion detection. Its strong performance across diverse datasets underscores adaptability, while low inference latency highlights readiness for real-world deployment. This balance between effectiveness, robustness, and efficiency positions MNTD as a strong candidate for next-generation intrusion detection systems.

## 7. Conclusion and future research work

This paper presented a robust framework for malicious network traffic detection that integrates CNN, BiLSTM, and multi-head attention mechanisms to capture both local and global traffic patterns. By employing contrastive learning for feature generalization and AWDV for efficient hyperparameter optimization, the model achieves strong robustness and adaptability. An adaptive loss function further enhances stability and performance across heterogeneous environments.

Experimental evaluations on benchmark datasets confirmed that the proposed framework achieves state-of-the-art detection results, highlighting its ability to generalize effectively across dynamic and diverse network conditions. The combination of packet-level and flow-level analysis ensures comprehensive detection of evolving and previously unseen threats, making the approach suitable for practical cybersecurity applications.

### Future Research Directions:

Looking ahead, several avenues can further extend this work:

- **Reinforcement Learning Integration:** We plan to incorporate reinforcement learning strategies to enable adaptive decision-making in real-time environments. RL will allow the model to dynamically update detection policies in response to evolving traffic behaviors, thereby enhancing resilience against zero-day attacks and reducing reliance on static training data.
- **Graph Neural Networks:** Leveraging graph-based representations of traffic flows will help capture complex inter-flow dependencies and improve detection of coordinated or stealthy attacks.
- **Efficiency in Resource-Constrained Environments:** Reducing computational overhead will be prioritized to support real-time deployment on edge and IoT devices without compromising detection accuracy.
- **Broader Validation:** Finally, testing the framework on a wider range of real-world datasets and application domains will further demonstrate its scalability and effectiveness.

### CRedit authorship contribution statement

**Mukhtar Ahmed:** Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Formal analysis, Data curation, Conceptualization. **Jinfu Chen:** Writing – review & editing, Supervision, Resources, Funding acquisition. **Ernest Akpaku:** Writing – review & editing, Visualization, Validation. **Ajmal Latif:** Writing – review & editing, Software, Methodology, Data curation.

### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

### Acknowledgments

This work was partly supported by the National Natural Science Foundation of China (NSFC) (Grant nos. 62172194, 62202206 and U1836116), the Natural Science Foundation of Jiangsu Province, China (Grant no. BK20220515), the China Postdoctoral Science Foundation, China (Grant no. 2021M691310), and Qinglan Project of Jiangsu Province, China.

### Data availability

Data will be made available on request.

### References

- Ahmad, R., Alsmadi, I., Alhamdani, W., Tawalbeh, L., 2022. A deep learning ensemble approach to detecting unknown network attacks. *J. Inf. Secur. Appl.* 67, 103196. <http://dx.doi.org/10.1016/j.jisa.2022.103196>.
- Ahmed, M., Chen, J., Akpaku, E., Latif, A., 2025a. BiRNN-SA: Context-aware malicious network traffic detection using self-attentive bidirectional RNNs. *Comput. Netw.* 272, 111658. <http://dx.doi.org/10.1016/j.comnet.2025.111658>.
- Ahmed, M., Chen, J., Akpaku, E., Sosu, R.N.A., 2025b. MTCR-AE: A Multiscale Temporal Convolutional Recurrent Autoencoder for unsupervised malicious network traffic detection. *Comput. Netw.* 261, 111147. <http://dx.doi.org/10.1016/j.comnet.2025.111147>.
- Ahmed, M., Chen, J., Akpaku, E., Sosu, R.N.A., Latif, A., 2024. DELM: Deep ensemble learning model for anomaly detection in malicious network traffic-based adaptive feature aggregation and network optimization. *ACM Trans. Priv. Secur.* 27 (4), <http://dx.doi.org/10.1145/3690637>.
- Akpaku, E., Chen, J., Ahmed, M., Sosu, R.N.A., Agbenyegah, F.K., Louis, D.K., 2025. eBiTCN: Efficient bidirectional temporal convolution network for encrypted malicious network traffic detection. *J. Comput. Secur.* 33 (3), 180–211. <http://dx.doi.org/10.1177/0926227X251326282>.
- Anitha, T., Aanjanakumar, S., Poonkuntran, S., Nayyar, A., 2023. A novel methodology for malicious traffic detection in smart devices using Bi-LSTM-CNN-dependent deep learning methodology. *Neural Comput. Appl.* 35, 20319–20338. <http://dx.doi.org/10.1007/s00521-023-08818-0>.
- Bautista, E., Brisson, L., Bothorel, C., Smits, G., 2024. MAD: Multi-scale anomaly detection in link streams. In: *Proceedings of the 17th ACM International Conference on Web Search and Data Mining. WSDM '24*, Association for Computing Machinery, New York, NY, USA, pp. 38–46. <http://dx.doi.org/10.1145/3616855.3635834>.
- Chai, Y., Chen, X., Qiu, J., Du, L., Xiao, Y., Feng, Q., Ji, S., Tian, Z., 2024. MalFSCIL: A few-shot class-incremental learning approach for malware detection. *Trans. Inf. Forensics Secur.* 20, 2999–3014. <http://dx.doi.org/10.1109/TIFS.2024.3516565>.
- Chen, J., Chen, Y., Cai, S., Yin, S., Zhao, L., Zhang, Z., 2023a. An optimized feature extraction algorithm for abnormal network traffic detection. *Future Gener. Comput. Syst.* 149, 330–342. <http://dx.doi.org/10.1016/j.future.2023.07.039>.
- Chen, A., Fu, Y., Zheng, X., Lu, G., 2022. An efficient network behavior anomaly detection using a hybrid DBN-LSTM network. *Comput. Secur.* 114, 102600. <http://dx.doi.org/10.1016/j.cose.2021.102600>.
- Chen, J., Lv, T., Cai, S., Song, L., Yin, S., 2023b. A novel detection model for abnormal network traffic based on bidirectional temporal convolutional network. *Inf. Softw. Technol.* 157, 107166. <http://dx.doi.org/10.1016/j.infsof.2023.107166>.
- Chen, J., Song, L., Cai, S., Xie, H., Yin, S., Ahmad, B., 2023c. TLS-MHSA: An efficient detection model for encrypted malicious traffic based on multi-head self-attention mechanism. *ACM Trans. Priv. Secur.* 26, <http://dx.doi.org/10.1145/3613960>.
- Chou, D., Jiang, M., 2021. A survey on data-driven network intrusion detection. *ACM Comput. Surv.* 54, <http://dx.doi.org/10.1145/3472753>.
- Elmasry, W., Akbulut, A., Zaim, A.H., 2020. Evolving deep learning architectures for network intrusion detection using a double PSO metaheuristic. *Comput. Netw.* 168, 107042. <http://dx.doi.org/10.1016/j.comnet.2019.107042>.
- Fang, Y., Ergüt, S., Patras, P., 2022. SDGNet: A handover-aware spatiotemporal graph neural network for mobile traffic forecasting. *IEEE Commun. Lett.* 26 (3), 582–586. <http://dx.doi.org/10.1109/LCOMM.2022.3141238>.
- Fang, Y., Li, K., Zheng, R., Liao, S., Wang, Y., 2021. A communication-channel-based method for detecting deeply camouflaged malicious traffic. *Comput. Netw.* 197, <http://dx.doi.org/10.1016/j.comnet.2021.108297>.
- Fu, C., Li, Q., Shen, M., Xu, K., 2021. Realtime robust malicious traffic detection via frequency domain analysis. In: *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. CCS '21*, Association for Computing Machinery, New York, NY, USA, pp. 3431–3446. <http://dx.doi.org/10.1145/3460120.3484585>.
- Fu, C., Li, Q., Xu, K., 2024. Flow interaction graph analysis: Unknown encrypted malicious traffic detection. *IEEE/ACM Trans. Netw.* 32 (4), 2972–2987. <http://dx.doi.org/10.1109/TNET.2024.3370851>.
- Gu, Y., Shao, Z., Qin, L., Lu, W., Li, M., 2019. A deep learning framework for cycling maneuvers classification. *IEEE Access* 7, 28799–28809. <http://dx.doi.org/10.1109/ACCESS.2019.2898852>.
- Hnamte, V., Hussain, J., 2023. DCNNBiLSTM: An efficient hybrid deep learning-based intrusion detection system. *Telemat. Inform. Rep.* 10, 100053. <http://dx.doi.org/10.1016/j.teler.2023.100053>.
- Hu, Z., Wang, R., Zhao, L., 2023. CLNet: Contrastive learning network for zero-day attack detection. *Neurocomputing* 545, 126451. <http://dx.doi.org/10.1016/j.neucom.2023.126451>.
- Kamal, H., Mashaly, M., 2024. Advanced hybrid transformer-CNN deep learning model for effective intrusion detection systems with class imbalance mitigation using resampling techniques. *Future Internet* 16 (12), 481. <http://dx.doi.org/10.3390/fi16120481>.
- Koroniotis, N., Moustafa, N., Sitnikova, E., Turnbull, B., 2019. Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset. *Future Gener. Comput. Syst.* 100, 779–796. <http://dx.doi.org/10.1016/j.future.2019.05.041>.
- Kumar, S., Janet, B., 2021. Distinguishing malicious programs based on visualization and hybrid learning algorithms. *Comput. Netw.* 201, 108595. <http://dx.doi.org/10.1016/j.comnet.2021.108595>.
- Kumar, S., Janet, B., 2022. DTMIC: Deep transfer learning for malware image classification. *J. Inf. Secur. Appl.* 64, 103063. <http://dx.doi.org/10.1016/j.jisa.2021.103063>.
- Kumar, S., Janet, B., Neelakantan, S., 2022. Identification of malware families using stacking of textual features and machine learning. *Expert Syst. Appl.* 208, 118073. <http://dx.doi.org/10.1016/j.eswa.2022.118073>.
- Kumar, S., Janet, B., Neelakantan, S., 2024. IMCNN: Intelligent Malware Classification using Deep Convolution Neural Networks as Transfer learning and ensemble learning in honeypot enabled organizational network. *Comput. Commun.* 216, 16–33. <http://dx.doi.org/10.1016/j.comcom.2023.12.036>.
- Kumar, S., Kumar, A., 2024. Image-based malware detection based on convolution neural network with autoencoder in Industrial Internet of Things using Software Defined Networking Honeypot. *Eng. Appl. Artif. Intell.* 133, 108374. <http://dx.doi.org/10.1016/j.engappai.2024.108374>.
- Kumar, S., Panda, K., 2023. SDIF-CNN: Stacking deep image features using fine-tuned convolution neural network models for real-world malware detection and classification. *Appl. Soft Comput.* 146, 110676. <http://dx.doi.org/10.1016/j.asoc.2023.110676>.
- Lei, S., Xia, C., Li, Z., Li, X., Wang, T., 2021. HNN: A novel model to study the intrusion detection based on multi-feature correlation and temporal-spatial analysis. *IEEE Trans. Netw. Sci. Eng.* 8, 3257–3274. <http://dx.doi.org/10.1109/TNSE.2021.3109644>.

- Li, Y., Zhou, F., Ma, J., 2024. GraphCL-ID: Graph contrastive learning for unsupervised intrusion detection in IoT networks. In: *Proceedings of the 31st Network and Distributed System Security Symposium. NDSS, Internet Society*.
- Lu, W., Yi, Z., Liu, W., Gu, Y., Rui, Y., Ran, B., 2020. Efficient deep learning based method for multi-lane speed forecasting: a case study in Beijing. *IET Intell. Transp. Syst.* 14 (14), 2073–2082. <http://dx.doi.org/10.1049/iet-its.2020.0410>.
- MontazeriShatoori, M., Davidson, L., Kaur, G., Lashkari, A.H., 2020. Detection of DoH tunnels using time-series classification of encrypted traffic. In: 2020 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress. DASC/PiCom/CBDCom/CyberSciTech, pp. 63–70, URL <https://api.semanticscholar.org/CorpusID:226852987>.
- Moustafa, N., Slay, J., 2015. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: 2015 Military Communications and Information Systems Conference. MilCIS, pp. 1–6. <http://dx.doi.org/10.1109/MilCIS.2015.7348942>.
- Naem, A., Khan, M.A., Khattak, A.A., et al., 2025. Efficient IoT intrusion detection with an improved attention-based CNN-BiLSTM architecture. *arXiv preprint arXiv:2503.19339*.
- Nguyen, T.M., Lee, S., 2024. Multi-view learning for intrusion detection: Cross-attention on payload and statistical features. *Comput. Secur.* 132, 103241. <http://dx.doi.org/10.1016/j.cose.2024.103241>.
- Ren, F., Jiang, Z., Liu, J., 2019. A bi-directional LSTM model with attention for malicious URL detection. In: *Proceedings of 2019 IEEE 4th Advanced Information Technology, Electronic and Automation Control Conference. IAEAC 2019, Institute of Electrical and Electronics Engineers Inc.*, pp. 300–305. <http://dx.doi.org/10.1109/IAEAC47372.2019.8997947>.
- Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: *International Conference on Information Systems Security and Privacy*. URL <https://api.semanticscholar.org/CorpusID:4707749>.
- Shen, S., Cai, C., Li, Z., Shen, Y., Wu, G., Yu, S., 2024. Deep Q-network-based heuristic intrusion detection against edge-based SLoT zero-day attacks. *Appl. Soft Comput.* 150, 111080. <http://dx.doi.org/10.1016/j.asoc.2023.111080>.
- Shen, S., Xie, L., Zhang, Y., Wu, G., Zhang, H., Yu, S., 2023. Joint differential game and double deep Q-networks for suppressing malware spread in industrial internet of things. *IEEE Trans. Inf. Forensics Secur.* 18, 5302–5315. <http://dx.doi.org/10.1109/TIFS.2023.3307956>.
- Song, B., Liu, Y., Fang, J., Liu, W., Zhong, M., Liu, X., 2024. An optimized CNN-BiLSTM network for bearing fault diagnosis under multiple working conditions with limited training samples. *Neurocomputing* 574, 127284. <http://dx.doi.org/10.1016/J.NEUCOM.2024.127284>.
- Sun, P., Liu, P., Li, Q., Liu, C., Lu, X., Hao, R., Chen, J., 2020a. DL-IDS: Extracting features using CNN-LSTM hybrid network for intrusion detection system. *Secur. Commun. Netw.* 2020 (1), 8890306. <http://dx.doi.org/10.1155/2020/8890306>.
- Sun, P., Liu, P., Li, Q., Liu, C., Lu, X., Hao, R., Chen, J., 2020b. DL-IDS: Extracting features using CNN-LSTM hybrid network for intrusion detection system. *Secur. Commun. Netw.* 2020, <http://dx.doi.org/10.1155/2020/8890306>.
- Tao, L., Gu, Y., Lu, W., Rui, X., Zhou, T., Ding, Y., 2020. An attention-based approach for traffic conditions forecasting considering spatial-temporal features. In: 2020 IEEE 5th International Conference on Intelligent Transportation Engineering. ICITE, pp. 117–122. <http://dx.doi.org/10.1109/ICITE50838.2020.9231367>.
- Wang, Y., 2024. Network anomaly traffic detection using WGAN-CNN-BiLSTM in big data cloud-edge collaborative computing environment. *J. Inf. Process. Syst.* 20 (3), 375–390.
- Xu, L., Cao, M., Song, B., 2022. A new approach to smooth path planning of mobile robot based on quartic Bezier transition curve and improved PSO algorithm. *Neurocomputing* 473, 98–106. <http://dx.doi.org/10.1016/J.NEUCOM.2021.12.016>.
- Yu, F., Tian, B., Yu, H., Ren, J., 2024a. An intrusion detection system based on GWO-CNN-BiLSTM. In: Yang, L. (Ed.), *Seventh International Conference on Advanced Electronic Materials, Computers, and Software Engineering (AEMCSE 2024)*. Vol. 13229, International Society for Optics and Photonics, SPIE, p. 132292C. <http://dx.doi.org/10.1117/12.3038081>.
- Yu, S., Wang, X., Shen, Y., Wu, G., Yu, S., Shen, S., 2024b. Novel intrusion detection strategies with optimal hyper parameters for industrial internet of things based on stochastic games and double deep Q-networks. *IEEE Internet Things J.* 11 (17), 29132–29145. <http://dx.doi.org/10.1109/JIOT.2024.3406386>.
- Yu, S., Zhai, R., Shen, Y., Wu, G., Zhang, H., Yu, S., Shen, S., 2024c. Deep Q-network-based open-set intrusion detection solution for industrial internet of things. *IEEE Internet Things J.* 11 (7), 12536–12550. <http://dx.doi.org/10.1109/JIOT.2023.3333903>.
- Zhang, W., Liu, F., Chen, X., 2023a. Transformer-based intrusion detection for network traffic using positional encoding and self-attention. *IEEE Trans. Netw. Serv. Manag.* 20 (1), 112–125. <http://dx.doi.org/10.1109/TNSM.2023.1001234>.
- Zhang, B., Liu, Z., Jia, Y., Ren, J., Zhao, X., 2018. Network intrusion detection method based on PCA and Bayes algorithm. *Secur. Commun. Netw.* 2018 (1), 1914980. <http://dx.doi.org/10.1155/2018/1914980>.
- Zhang, J., Zhang, X., Liu, Z., Fu, F., Jiao, Y., Xu, F., 2023b. A network intrusion detection model based on BiLSTM with multi-head attention mechanism. *Electronics* 12 (19), 4170. <http://dx.doi.org/10.3390/electronics12194170>.
- Zhu, W., Liu, X., Liu, Y., Shen, Y., Gao, X.-Z., Shen, S., 2025. RT-A3C: Real-time Asynchronous Advantage Actor-Critic for optimally defending malicious attacks in edge-enabled Industrial Internet of Things. *J. Inf. Secur. Appl.* 91, 104073. <http://dx.doi.org/10.1016/j.jisa.2025.104073>.



**Mukhtar Ahmed** received his B.Sc. degree in Computer Science from BUITEMS, Pakistan, and the MS degree in Computer Science from ILMA University, Pakistan. He is currently pursuing a Ph.D. degree at the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China. His research interests include malicious network detection, network security, deep learning, and cloud security. He is a member of the Association for Computing Machinery (ACM).



**Jinfu Chen** received his Ph.D. degree in Computer Science and Technology from Huazhong University of Science and Technology, Wuhan, China, in 2009. He is currently a full professor at the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China. His major research interests include Software Testing, Software Security, and Trusted Software. He has published more than 80 papers in some famous journals or conferences. He is a member of the IEEE and the ACM, and a member of the China Computer Federation.



**Ernest Akpaku** received his M.Phil. degree in management information systems from the University of Ghana. He is currently pursuing a Ph.D. degree in computer science and technology from the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China. Ernest's research areas include network security, malicious network traffic detection, vulnerability detection, and deep learning. He is a member of the Association for Computing Machinery (ACM).



**Ajmal Latif** received his Bachelor's degree in Telecommunication from the Institute of Business and Technology, Pakistan, and his Master's degree in Computer Science from ILMA University, Pakistan. He is currently serving as Deputy Director of IT at the Directorate of Information Technology, Lasbela University of Agriculture, Water and Marine Sciences, Pakistan. His research interests include network security and cybersecurity. He is also a member of the Association for Computing Machinery (ACM).